Consumer Sentiment Analysis of Apple and Google Products Using NLP

Author: Diana Byegon

Project Type: Multiclass Text Classification

Dataset: CrowdFlower Twitter Sentiment Dataset on Apple and Google Products

Primary Technique: Natural Language Processing (NLP)

Advanced Components: Feature engineering, stratified cross-validation, hyperparameter tuning using GridSearchCV, ensemble learning (Random Forest), comparative model evaluation, confusion matrix analysis, classification reports, and model interpretability through feature importance analysis.

Executive Summary

Social media platforms provide organizations with valuable insights into customer perceptions, product experiences, and emerging concerns. This project applied Natural Language Processing (NLP) techniques to analyze customer sentiment in Twitter conversations related to Apple and Google products.

The analysis revealed that Neutral sentiment dominated customer conversations, while Positive sentiment was the second most common category. Product-level analysis showed that iPhone-related tweets exhibited the highest proportion of negative sentiment, indicating greater levels of customer dissatisfaction compared to other products. Further analysis identified key sentiment drivers, with positive sentiment associated with words such as *smart*, *cool*, and *great*, while negative sentiment was linked to terms such as *headache*, *fail*, and *hate*.

Several machine learning models were evaluated, including Logistic Regression, LinearSVC, and Random Forest. CountVectorizer combined with Logistic Regression achieved the strongest overall performance, attaining a weighted F1-score of 0.65 on unseen test data and outperforming more complex alternatives.

Based on these findings, organizations should implement continuous sentiment monitoring to identify emerging customer concerns, prioritize products associated with higher levels of negative sentiment, and leverage positive customer feedback to inform product development and marketing strategies.

1. Business Understanding

1.1 Business Context

Social media is one of the fastest channels through which customers express opinions about technology products. During product launches, conferences, updates, and service issues, consumers

often share reactions publicly on Twitter. For companies such as Apple and Google, these tweets can contain valuable signals about customer satisfaction, product perception, and potential customer experience issues.

However, manually reviewing thousands of tweets is slow, expensive, and difficult to scale. Product marketing and customer experience teams need a way to quickly separate positive, negative, and neutral feedback so they can understand public perception and respond to emerging concerns.

1.2 Business Problem

Product marketing and customer experience teams need timely insights into how customers perceive Apple and Google products. While social media contains a wealth of customer feedback, manually reviewing thousands of tweets is time-consuming, resource-intensive, and impractical. Without an automated sentiment analysis system, organizations may struggle to:

(i) Detect negative customer sentiment early.

(ii) Identify emerging product issues.

(iii) Measure public reaction to product launches and marketing campaigns.

(iv) Understand the key factors driving customer satisfaction and dissatisfaction.

This project aims to address this challenge by developing a machine learning model capable of automatically classifying the sentiment expressed in tweets related to Apple and Google products.

1.3 Stakeholders

Primary stakeholders: Product Marketing Managers at Apple and Google.
They are responsible for tracking customer perception, measuring campaign and launch reception, and understanding how customers talk about products.

Secondary stakeholders: Customer Experience Teams.
They can use sentiment classification to identify negative tweets that may require follow-up or deeper investigation.

Strategic stakeholders: Brand and Communications Teams.
They can use sentiment trends to understand public reaction and protect brand reputation.

1.4 Project Goal

The goal of this project is to develop a Natural Language Processing (NLP) model capable of automatically classifying sentiment expressed in tweets related to Apple and Google products.

The model will categorize tweets according to their sentiment and provide insights into the language and factors that influence customer opinions.

1.5 Business Questions

1. **What overall sentiment are customers expressing toward Apple and Google products on Twitter?**
2. **Can tweet content be used to accurately identify customer sentiment?**
3. **What words and phrases most strongly influence positive and negative customer sentiment?**

1.6 Success Metrics

Business success:
The project is successful if it creates a model that helps stakeholders monitor sentiment more efficiently and provides actionable insight into customer language.

Technical success:
The primary metric is **weighted F1-score** because the dataset contains imbalanced sentiment classes. Weighted F1 balances precision and recall while accounting for class size.

Secondary metrics include accuracy, precision, recall, and confusion matrix results.

2. Data Understanding

2.1 Data Source

The dataset used in this project is the Twitter Sentiment Dataset, originally collected through CrowdFlower and made available through Data.world. The dataset contains tweets discussing Apple and Google products and includes manually assigned sentiment labels.

Human annotators reviewed each tweet and classified the sentiment expressed toward a brand or product.This dataset is appropriate for the project because it contains real-world customer opinions that can be used to train a machine learning model

2.2 Data Overview

2.2.1 Importing Libraries

Before exploring the dataset, the necessary Python libraries must be imported to support data manipulation, visualization, and exploratory analysis.

2.2.2 Loading Data

In [135…

```
df = pd.read_csv( r'Data\judge-1377884607_tweet_product_company.csv',encoding='latin1')
df.head()
```

Out[135…

	tweet_text	emotion_in_tweet_is_directed_at	is_there_an_emotion_directed_at_a_brand_or_product
0	.@wesley83 I have a 3G iPhone. After 3 hrs tweeting at #RISE_Austin, it was dead! I need to upgrade. Plugin stations at #SXSW.	iPhone	Negative emotion
1	@jessedee Know about @fludapp ? Awesome iPad/iPhone app that you'll likely appreciate for its design. Also, they're giving free Ts at #SXSW	iPad or iPhone App	Positive emotion
2	@swonderlin Can not wait for #iPad 2 also. They should sale them down at #SXSW.	iPad	Positive emotion
3	@sxsw I hope this year's festival isn't as crashy as this year's iPhone app. #sxsw	iPad or iPhone App	Negative emotion
4	@sxtxstate great stuff on Fri #SXSW: Marissa Mayer (Google), Tim O'Reilly (tech books/conferences) & Matt Mullenweg (Wordpress)	Google	Positive emotion

The dataset consists of three main variable; tweet text, the product or brand the tweet is directed at, and a sentiment label. The tweet text will be the main predictor, while the sentiment label will be the target variable for supervised learning.

2.2.3 Data Shape

Understanding the size of the dataset helps determine whether sufficient observations are available for model training and evaluation.

In [4]:
```
# Dimensions
df.shape
```

Out[4]:
```
(9093, 3)
```

The dataset contains 9,093 rows and 3 columns.

Rows represent individual tweets. Columns represent tweet features and sentiment labels.

With over 9,000 observations, the dataset provides a sufficiently large sample of customer opinions to support machine learning analysis.

2.2.4 Data Information

Examining the dataset structure helps identify data types and missing values that may require attention during data cleaning.

In [5]:
```
# Data info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9093 entries, 0 to 9092
Data columns (total 3 columns):
 #   Column                                              Non-Null Count  Dtype 
---  ------                                              --------------  ----- 
 0   tweet_text                                          9092 non-null   object
 1   emotion_in_tweet_is_directed_at                     3291 non-null   object
 2   is_there_an_emotion_directed_at_a_brand_or_product  9093 non-null   object
dtypes: object(3)
memory usage: 213.2+ KB
```

The dataset contains both complete and incomplete fields. While sentiment labels are fully populated, some observations contain missing product information.

Missing product information may limit product-level analysis but does not prevent sentiment classification because the primary predictor is tweet text.

2.2.5 Descriptive Statistics

Descriptive statistics provide a summary of the dataset and help identify potential data quality issues.

In [6]:
```
# Summary Statistics
df.describe(include='all')
```

Out[6]:

	tweet_text	emotion_in_tweet_is_directed_at	is_there_an_emotion_directed_at_a_brand_or_product
count	9092	3291	9093
unique	9065	9	4
top	RT @mention Marissa Mayer: Google Will Connect the Digital & Physical Worlds Through Mobile -	iPad	No emotion toward brand or product

	tweet_text	emotion_in_tweet_is_directed_at	is_there_an_emotion_directed_at_a_brand_or_product
	{link} #sxsw		
freq	5	946	5389

The dataset contains tweet text, the product or brand the tweet is directed at, and a sentiment label. The tweet text will be the main predictor, while the sentiment label will be the target variable for supervised learning.

2.3 Data Quality Assessment

Assessing the overall quality of the dataset helps identify missing values, duplicate records, and potential issues that may affect model performance and interpretation.

Understanding these issues early allows us to develop an appropriate data cleaning strategy while maintaining transparency and reproducibility throughout the project.

2.3.1 Missing Values

Missing values can reduce data quality and potentially impact downstream analysis and model performance. This step identifies the extent of missing information within each feature.

In [7]:
```
# Checking for missing values
missing_values = df.isnull().sum()
missing_values
```

Out[7]:
```
tweet_text                                            1
emotion_in_tweet_is_directed_at                    5802
is_there_an_emotion_directed_at_a_brand_or_product    0
dtype: int64
```

The output shows the number of missing observations in each feature.The majority of missing values occur in the product-related feature, while the tweet text and sentiment label fields contain little to no missing data.

Since sentiment labels represent the target variable, missing values in this field would significantly reduce the dataset's usefulness for supervised learning. Missing product information may affect product-level analysis but is less critical for sentiment classification because tweet text remains the primary predictor.

2.3.2 Duplicate Records

Duplicate observations can bias model training and inflate performance metrics if not identified and addressed.

In [8]:
```
# Check for duplicate records

duplicates = df.duplicated().sum()

print(f"Number of duplicate records: {duplicates}")
```

```
Number of duplicate records: 22
```

Duplicate tweets may cause the model to learn repetitive patterns that do not accurately represent real-world customer sentiment. Removing duplicates helps improve the reliability of the analysis.

2.3.3 Data Types

Verifying data types ensures that each variable is stored in an appropriate format for analysis and modeling.

In [10]:
```
# data types

df.dtypes
```

Out[10]:
```
tweet_text                                            object
emotion_in_tweet_is_directed_at                       object
is_there_an_emotion_directed_at_a_brand_or_product    object
dtype: object
```

All variables are currently stored as object (string) data types, which is expected because the dataset consists primarily of textual information.

2.3.4 Data Quality Summary

The data quality assessment revealed several important observations:

(i) The dataset contains 9,093 tweets.

(ii) The sentiment label column contains no missing values, making it suitable for supervised learning.

(iii) The product-related feature contains a substantial number of missing values.

(iv) The tweet text feature contains very few missing observations.

(v) Duplicate records may exist and should be addressed during data cleaning.

(vi) All variables are stored as text-based features.

Implications for the Project

The dataset remains highly suitable for sentiment classification because the target variable is complete and the tweet text field is largely intact.

The primary data quality challenge involves missing product information. However, since the objective of the project is to classify sentiment based on tweet content, this issue is not expected to significantly impact model performance.

2.3.5 Data Limitations

The dataset is useful for a proof-of-concept NLP sentiment classifier, but it has limitations:

Class imbalance: Negative and unclear tweets are much less common than neutral/no-emotion tweets.

Missing product labels: Many tweets do not specify the product or brand they are directed at.

Historical Twitter data: Language patterns may differ from current social media behavior.

Human labeling subjectivity: Sentiment labels depend on human raters and may include ambiguity.

Short text complexity: Tweets are brief and may contain sarcasm, abbreviations, hashtags, and informal language.

3. Data Cleaning

The purpose of data cleaning is to improve data quality by addressing missing values, duplicate records, and inconsistencies in the text data. Since machine learning models rely heavily on data quality, cleaning the dataset is an essential step before exploratory analysis and model development.

Data Quality Issues Identified

The Data Understanding phase identified the following issues:

(i) Missing values in the product feature

(ii) Missing values in tweet text

(iii) Duplicate records

(iv) Unstructured tweet text

(v) Inconsistent text formatting

These issues will be addressed in this section.

3.1 Renaming Columns

Long column names can make code difficult to read and maintain. Renaming columns to concise and descriptive names improves readability and simplifies analysis throughout the project.

In [11]:
```
# Rename columns

df.rename(
    columns={
        'tweet_text': 'tweet',
        'emotion_in_tweet_is_directed_at': 'product',
        'is_there_an_emotion_directed_at_a_brand_or_product': 'sentiment'
    },
    inplace=True
```

```
)

# Verify changes
df.head()
```

Out[11]:

	tweet	product	sentiment
0	.@wesley83 I have a 3G iPhone. After 3 hrs tweeting at #RISE_Austin, it was dead! I need to upgrade. Plugin stations at #SXSW.	iPhone	Negative emotion
1	@jessedee Know about @fludapp ? Awesome iPad/iPhone app that you'll likely appreciate for its design. Also, they're giving free Ts at #SXSW	iPad or iPhone App	Positive emotion
2	@swonderlin Can not wait for #iPad 2 also. They should sale them down at #SXSW.	iPad	Positive emotion
3	@sxsw I hope this year's festival isn't as crashy as this year's iPhone app. #sxsw	iPad or iPhone App	Negative emotion
4	@sxtxstate great stuff on Fri #SXSW: Marissa Mayer (Google), Tim O'Reilly (tech books/conferences) & Matt Mullenweg (Wordpress)	Google	Positive emotion

3.2 Missing Values Treatment

Identified missing Tweet text and Missing product information in the data understanding phase. Cleaning the missing values as they can reduce data quality and may affect analysis and modeling results.

3.2.1 Handling Missing Tweet Text

Tweet text serves as the primary predictor variable. Records without tweet text cannot contribute meaningful information to the sentiment classification task.

In [12]:
```
# Remove rows with missing tweet text

df = df.dropna(subset=['tweet'])
```

verification

In [13]:
```
# Verify removal

df['tweet'].isnull().sum()
```

Out[13]: np.int64(0)

Rows containing missing tweet text have been removed from the dataset.

3.2.2 Handling Missing Product Information

A large number of observations contain missing product information. Removing these observations would result in unnecessary loss of potentially valuable sentiment information. Missing values will be replaced with the category "Unknown".

In [14]:
```
# Replace missing product values
```

```
df['product'] = (
    df['product']
    .fillna('Unknown')
)
```

Verification

In [15]:
```
# Verify replacement

df['product'].isnull().sum()
```

Out[15]: np.int64(0)

All missing product information has been replaced with the label "Unknown".

3.3 Duplicate Record Treatment

Duplicate rows can overrepresent repeated tweets and bias model training.

3.3.1 Removing Duplicate Records

In [16]:
```
# Remove duplicate records

df = df.drop_duplicates()
```

Verification

In [17]:
```
# Verify duplicates removed

df.duplicated().sum()
```

Out[17]: np.int64(0)

Duplicate observations have been successfully removed. Removing duplicates ensures that the analysis reflects unique customer opinions rather than repeated observations.

3.4 Text Standardization

Tweet text often contains inconsistent formatting that can create unnecessary variation during analysis. Standardizing text improves consistency and prepares the data for downstream NLP preprocessing.

3.4.1 Converting Tweet Text to Lowercase

This ensures same words are interparted consistently.

In [18]:
```
# Convert text to lowercase

df.loc[:, 'tweet'] = df['tweet'].str.lower()
```

3.4.2 Removing URLs

URLs generally do not contribute meaningful sentiment information.

In [20]:
```
# Remove URLs

df.loc [:, 'tweet'] = df['tweet'].apply(
    lambda x: re.sub(r'http\S+|www\S+', '', str(x))
)
```

Removing URLs reduces noise while preserving sentiment-related content.

3.4.3 Removing Twitter Mentions

User mentions do not contribute useful information for sentiment analysis.

In [21]:
```
# Remove mentions

df.loc[:, 'tweet'] = df['tweet'].apply(
    lambda x: re.sub(r'@\w+', '', str(x))
)
```

3.4.4 Removing Punctuation

Punctuation can introduce unnecessary variation into textual data.

In [22]:
```
import string

# Remove punctuation

df.loc[:, 'tweet'] = df['tweet'].apply(
    lambda x: x.translate(
        str.maketrans('', '', string.punctuation)
    )
)
```

3.4.5 Removing Numbers

Most numerical values do not directly contribute to sentiment detection.

In [23]:
```
# Remove numbers

df.loc[:,'tweet'] = df['tweet'].apply(
    lambda x: re.sub(r'\d+', '', str(x))
)
```

3.4.6 Removing Extra Whitespace

Extra spaces can create inconsistencies within textual data.

In [24]:
```
# Remove extra whitespace

df.loc[:,'tweet'] = df['tweet'].apply(
    lambda x: ' '.join(x.split())
)
```

3.4.7 Standardize Sentiment Labels

Consistent target labels make modeling and interpretation easier.

In [25]:
```
# Create an independent copy of the dataframe
df = df.copy()
sentiment_mapping = {
    'Positive emotion': 'Positive',
    'Negative emotion': 'Negative',
    'No emotion toward brand or product': 'Neutral',
    "I can't tell": 'Unclear'
}

df['sentiment'] = df['sentiment'].replace(sentiment_mapping)

df['sentiment'].value_counts()
```

Out[25]:
```
sentiment
Neutral     5375
Positive    2970
Negative     569
Unclear      156
Name: count, dtype: int64
```

Standardized labels improve readability and make it easier for stakeholders to interpret visualizations, model outputs, and business recommendations.

3.5 Data Cleaning Summary

The data cleaning process was conducted to improve data quality, consistency, and suitability for analysis and machine learning. Several issues identified during the Data Understanding phase were addressed through a systematic cleaning approach.

The following data cleaning steps were completed:

(i) Renamed columns to shorter and more descriptive names (tweet, product, and sentiment) to improve code readability and maintainability.

(ii) Standardized sentiment labels into four consistent categories: Positive, Negative, Neutral, and Unclear.

(iii) Removed observations with missing tweet text, as they contained no information useful for sentiment classification.

(iv) Replaced missing product information with "Unknown" to preserve valuable sentiment data while maintaining transparency regarding missing values.

(v) Identified and removed duplicate records to ensure each observation represented a unique customer opinion.

(vi) Standardized tweet text by: Converting all text to lowercase, Removing URLs, removing Twitter user mentions, removing punctuation, removing numerical values and removing extra whitespace.

These cleaning steps reduced noise, improved data consistency, and enhanced the overall quality of the dataset. The resulting dataset is more suitable for exploratory analysis and machine learning

while preserving the information necessary to answer the project's business questions.

4. Exploratory Data Analysis (EDA)

The purpose of Exploratory Data Analysis (EDA) is to uncover patterns, trends, and relationships within the dataset that can help answer the project's business questions.

By exploring the distribution of sentiment, product discussions, and tweet characteristics, we can gain a deeper understanding of customer perceptions toward Apple and Google products. These insights will also guide feature engineering and model development in later stages of the project.

4.1 Target Variable Analysis.

This section seeks to answer the Business Question 1 in regards to What overall sentiment are customers expressing toward Apple and Google products on Twitter.

4.1.1 Sentiment Class Distribution

Understanding the distribution of the target variable and examining sentiment frequencies provides an initial understanding of overall customer sentiment and helps identify potential class imbalance issues.

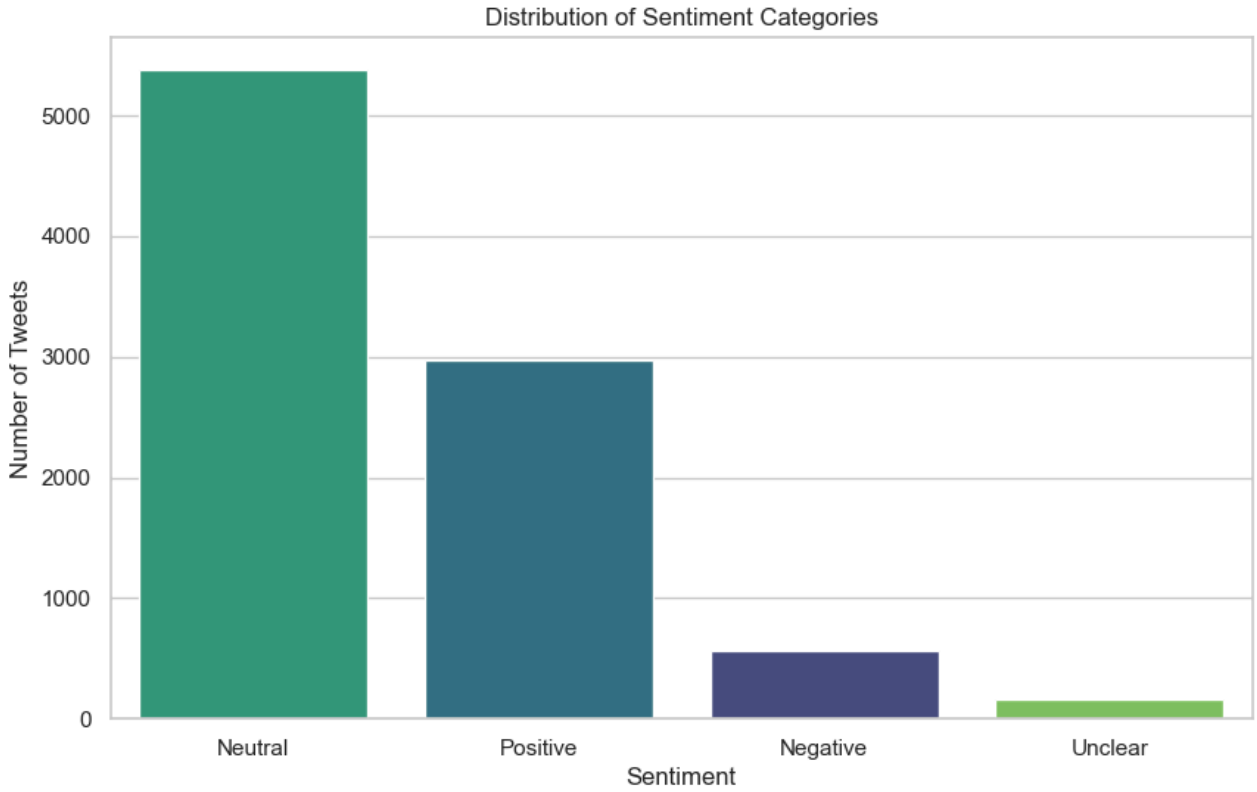
The code below calculates the frequency of each sentiment category.

In [26]:
```
# Display sentiment distribution

df['sentiment'].value_counts()
```

Out[26]:
```
sentiment
Neutral     5375
Positive    2970
Negative     569
Unclear      156
Name: count, dtype: int64
```

Sentiment Distribution by Categories

In [28]:
```
plt.figure(figsize=(10, 6))

sns.countplot(
    data=df,
    x='sentiment',
    order=df['sentiment'].value_counts().index,
    hue='sentiment',
    palette='viridis',
    legend=False
)

plt.title('Distribution of Sentiment Categories')
plt.xlabel('Sentiment')
plt.ylabel('Number of Tweets')

plt.show()
```



The dataset contains four sentiment categories with varying frequencies. The majority of tweets are classified as Neutral (5,375 tweets), followed by Positive sentiment (2,970 tweets). Negative sentiment represents a much smaller proportion of the dataset (569 tweets), while Unclear sentiment is the least represented category (156 tweets).

Insights The dominance of neutral tweets suggests that many customer conversations are informational rather than emotional. Among tweets expressing sentiment, positive opinions are considerably more common than negative ones, indicating generally favorable perceptions of Apple and Google products within this dataset.

For marketing and brand management teams, this suggests that public sentiment is largely neutral-to-positive, although the presence of negative sentiment highlights opportunities to identify customer concerns and improve user experiences.

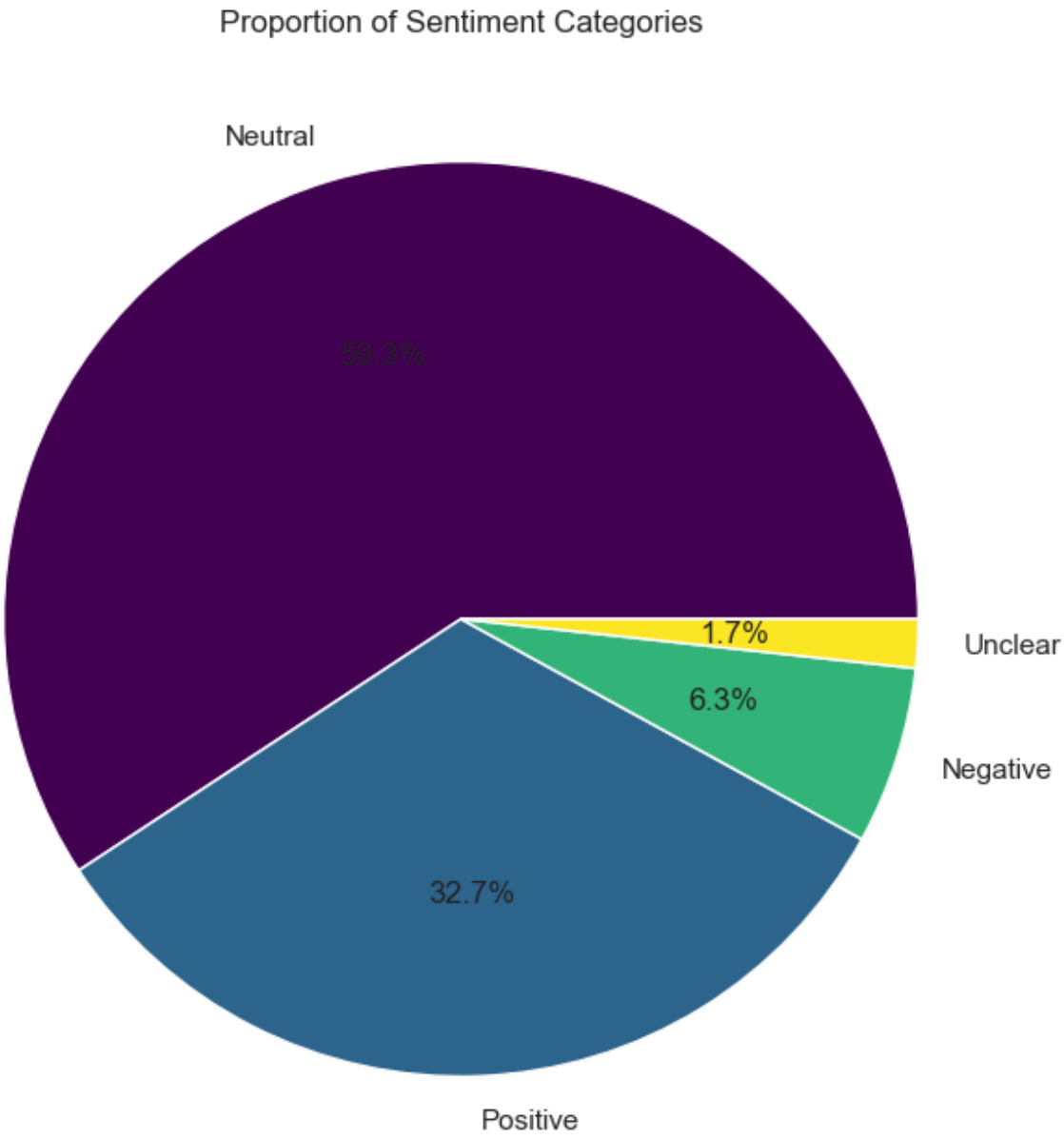
4.1.2 Sentiment Proportions

While frequency counts show the number of observations, percentage distributions provide a clearer understanding of the relative contribution of each sentiment category.

Proportion of Sentiment Categories

In [29]:
```
# Get sentiment counts
counts = df['sentiment'].value_counts()

# Generate viridis colors
colors = plt.cm.viridis(np.linspace(0, 1, len(counts)))

plt.figure(figsize=(8, 8))

counts.plot(
```

```
    kind='pie',
    autopct='%1.1f%%',
    colors=colors
)

plt.title('Proportion of Sentiment Categories')
plt.ylabel('')

plt.show()
```



The pie chart shows that Neutral sentiment accounts for 59.0% of all tweets, making it the dominant sentiment category. Positive sentiment represents 32.9% of tweets, while Negative sentiment accounts for only 6.3%. The Unclear category contributes just 1.7% of observations.

Insights

For stakeholders, this finding suggests that customer sentiment is largely positive; however, the smaller negative segment remains important because it may reveal product issues, customer

frustrations, or areas for improvement that could impact customer satisfaction and brand reputation.

4.1.3 Class Imbalance Assessment

The sentiment distribution reveals a clear class imbalance within the dataset. Neutral tweets account for approximately 59% of all observations, while Positive tweets represent 32.9%. In contrast, Negative and Unclear tweets make up only 6.3% and 1.7% of the dataset, respectively.

This means that the majority of observations belong to the Neutral and Positive classes, while Negative and Unclear sentiments are significantly underrepresented.

In an imbalanced dataset, a model may achieve high accuracy simply by predicting the majority classes more frequently. As a result, accuracy alone may provide a misleading assessment of model performance.

Insights

From a business perspective, correctly identifying Negative sentiment is particularly important because negative customer experiences often highlight product issues, dissatisfaction, or areas requiring immediate attention. Although Negative tweets represent a small proportion of the dataset, they may contain some of the most actionable insights for marketing and customer experience teams.

Implications for Modeling

To account for the observed class imbalance, Weighted F1-Score will be used as the primary evaluation metric. Unlike accuracy, Weighted F1-Score considers both precision and recall while accounting for the distribution of sentiment classes, providing a more balanced assessment of model performance.

4.2 Univariate Analysis

Having explored the target variable and established the overall sentiment distribution, the next step is to examine individual variables within the dataset. Understanding these variables independently provides valuable context for customer conversations and helps uncover patterns that may influence sentiment classification.

In the context of this project, univariate analysis will be used to explore key variables such as product mentions and tweet characteristics. Understanding these variables individually will help provide context for customer conversations and support the development of effective sentiment classification models.

4.2.1 Product Distribution

This is where we begin answering which products the customers are discussing frequently.

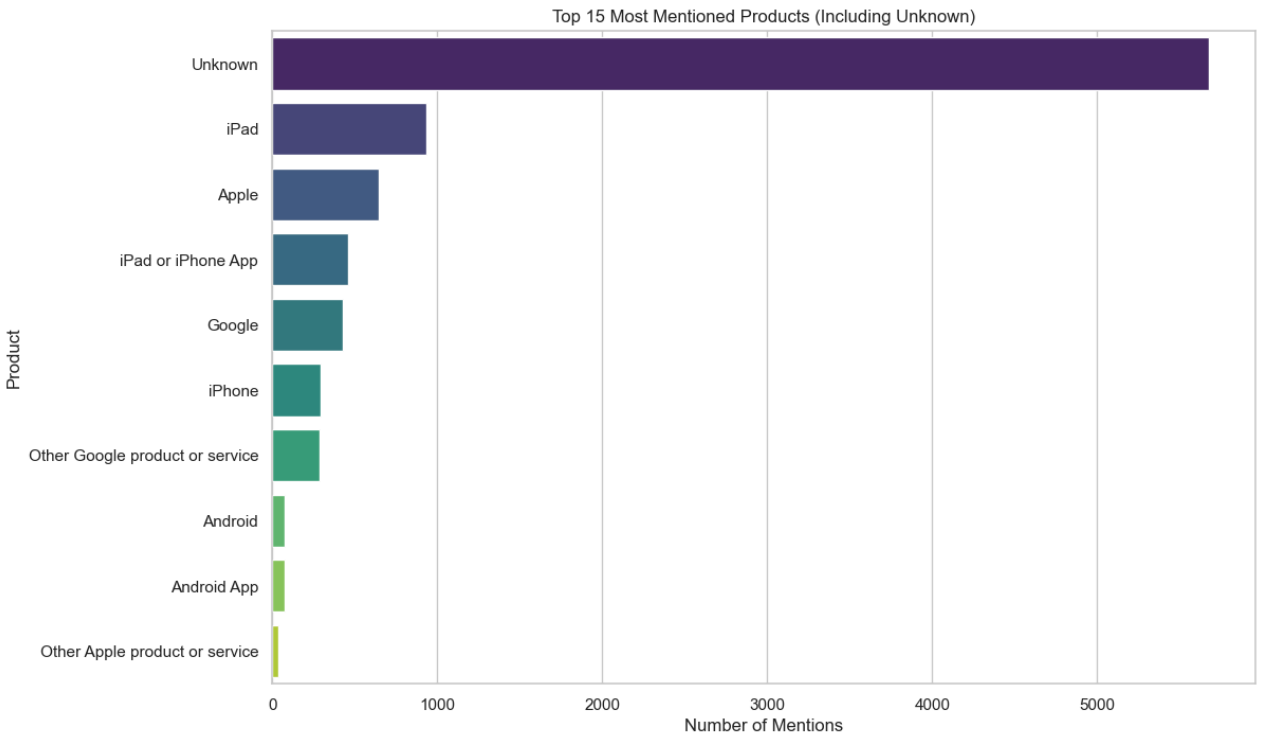
In [86]:
```
# Top product mentions
```

```
df['product'].value_counts().head(15)
```

Out[86]:
```
product
Unknown                            5678
iPad                                936
Apple                               648
iPad or iPhone App                  462
Google                              425
iPhone                              295
Other Google product or service     289
Android                              77
Android App                          76
Other Apple product or service       35
Name: count, dtype: int64
```

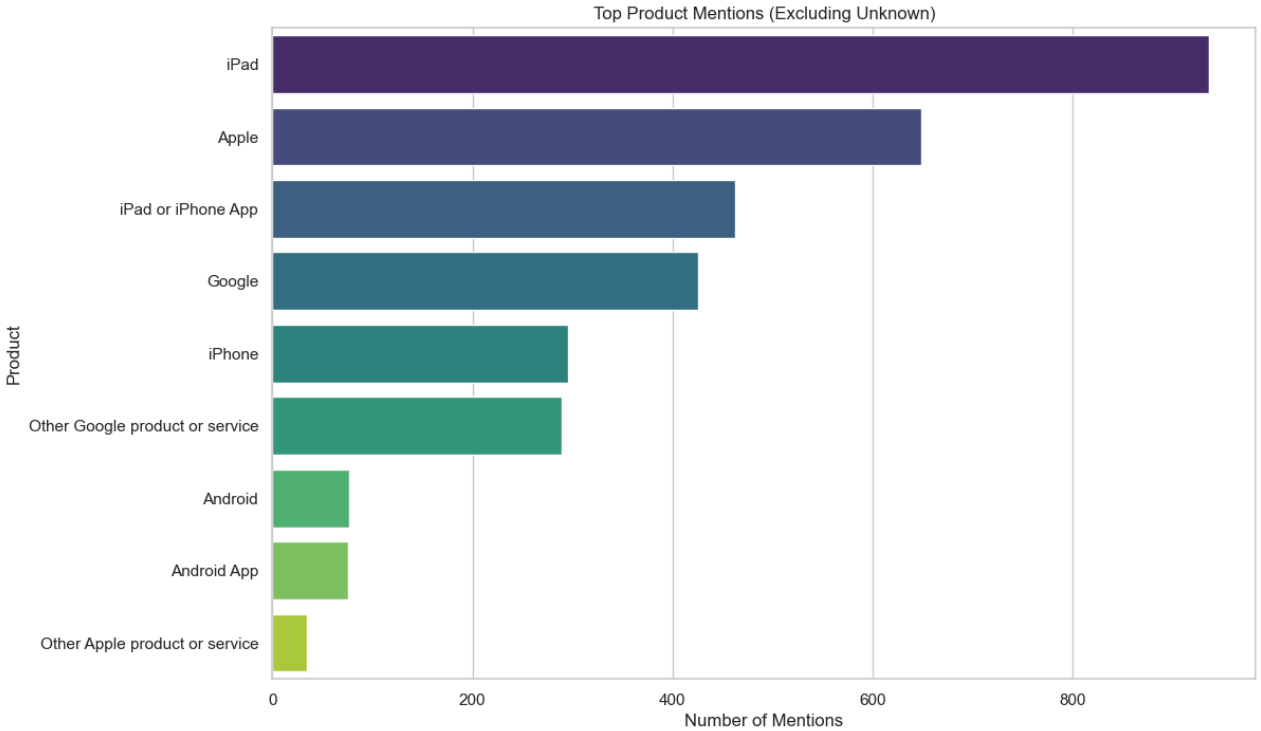
Product Distribution: Analysis 1 inlcudes The "Unknown" product category.

In [107…
```
product_counts = df['product'].value_counts().head(15)

plt.figure(figsize=(12,8))

sns.barplot(
    x=product_counts.values,
    y=product_counts.index,
    hue=product_counts.index,
    palette='viridis',
    legend=False
)

plt.title('Top 15 Most Mentioned Products (Including Unknown)')
plt.xlabel('Number of Mentions')
plt.ylabel('Product')

plt.show()
```



Product Distribution: Analysis 2 excludes The "Unknown" product category.

In [91]:
```
# Product distribution excluding Unknown

product_counts = (
    df[df['product'] != 'Unknown']['product']
    .value_counts()
    .head(15)
)

plt.figure(figsize=(12,8))

sns.barplot(
    x=product_counts.values,
    y=product_counts.index,
    hue=product_counts.index,
    palette='viridis',
    legend=False
)

plt.title('Top Product Mentions (Excluding Unknown)')
plt.xlabel('Number of Mentions')
plt.ylabel('Product')

plt.show()
```



The product distribution reveals that a substantial number of tweets are categorized as Unknown, indicating that many observations do not explicitly identify a specific Apple or Google product.

Among tweets with identifiable products, the iPad is the most frequently discussed product, followed by Apple, iPad or iPhone App, Google, and iPhone. Android-related products appear less frequently in the dataset.

Insights

The high number of unknown product references suggests that customers often discuss brands or experiences without explicitly mentioning a specific product. This highlights the importance of analyzing tweet content directly rather than relying solely on product identifiers.

Among identifiable products, Apple-related products appear more frequently than Google-related products, suggesting that Apple products generated a larger share of customer conversations within this dataset.

4.2.2 Tweet Length Distribution

Tweet length can influence sentiment classification performance because longer tweets often contain more contextual information than shorter tweets.Understanding the distribution of tweet lengths helps identify common communication patterns and informs feature engineering decisions later in the project.

In [30]:
```
# Create tweet length feature

df['tweet_length'] = df['tweet'].apply(len)

# Preview
df[['tweet', 'tweet_length']].head()
```

Out[30]:

	tweet	tweet_length
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	104
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	112
2	can not wait for ipad also they should sale them down at sxsw	61
3	i hope this years festival isnt as crashy as this years iphone app sxsw	71
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	107

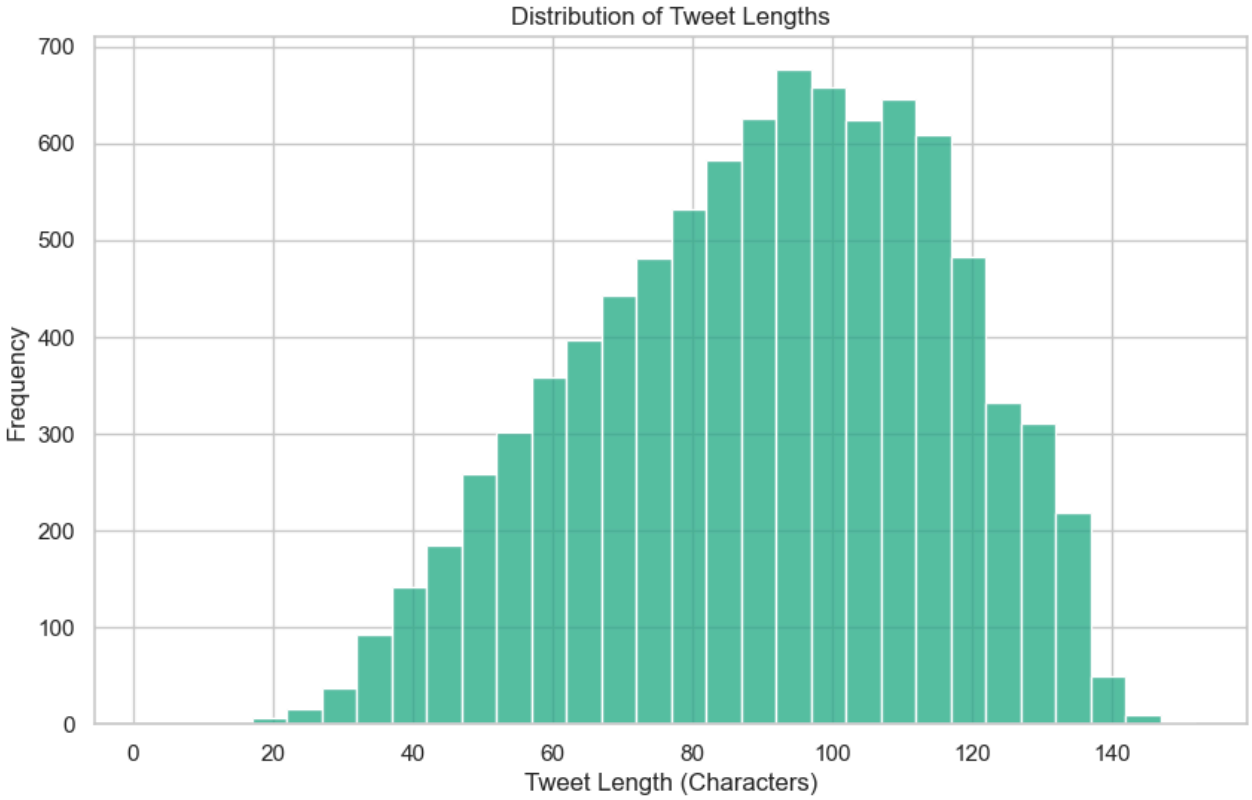
In [31]:
```
# Summary statistics

df['tweet_length'].describe()
```

Out[31]:
```
count    9070.000000
mean       89.957332
std        25.133610
min         2.000000
25%        72.000000
50%        92.000000
75%       109.000000
max       152.000000
Name: tweet_length, dtype: float64
```

The average tweet length is approximately 90 characters, with a median of 92 characters, indicating a fairly balanced distribution. Most tweets fall between 72 and 110 characters, suggesting that users typically express their opinions through short, concise messages. While a few tweets are extremely short or approach the character limit, tweet lengths are generally consistent across the dataset.

Tweet Length Distribution

In [32]:
```
plt.figure(figsize=(10,6))

sns.histplot(
    data=df,
    x='tweet_length',
    bins=30,
    color = plt.cm.viridis(0.6)
)

plt.title('Distribution of Tweet Lengths')
plt.xlabel('Tweet Length (Characters)')
plt.ylabel('Frequency')

plt.show()
```



The distribution of tweet lengths appears approximately bell-shaped, with most tweets ranging between 70 and 120 characters. The highest concentration of tweets falls around 90–110 characters, which aligns closely with the mean and median tweet lengths observed earlier. Very short and very long tweets are relatively uncommon.

The distribution does not appear heavily skewed and shows no evidence of excessive outliers, suggesting that tweet length is relatively consistent across observations and unlikely to require additional transformation prior to modeling.

4.2.3 Word Count Distribution

Word count measures the actual amount of language used by customers, making it particularly relevant for sentiment analysis.The purpose of this analysis is to examine the number of words used

in customer tweets. Word count provides insight into the amount of information contained within each tweet and may influence how sentiment is expressed.

Understanding word count patterns can also help identify whether customers typically communicate through short reactions or more detailed opinions.

Number of words contained in each tweet

In [33]:
```
# Create word count feature

df['word_count'] = df['tweet'].apply(
    lambda x: len(str(x).split())
)

# Preview
df[['tweet', 'word_count']].head()
```

Out[33]:

	tweet	word_count
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	21
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	19
2	can not wait for ipad also they should sale them down at sxsw	13
3	i hope this years festival isnt as crashy as this years iphone app sxsw	14
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	16

Summary statistics

In [34]:
```
# Summary statistics

df['word_count'].describe()
```

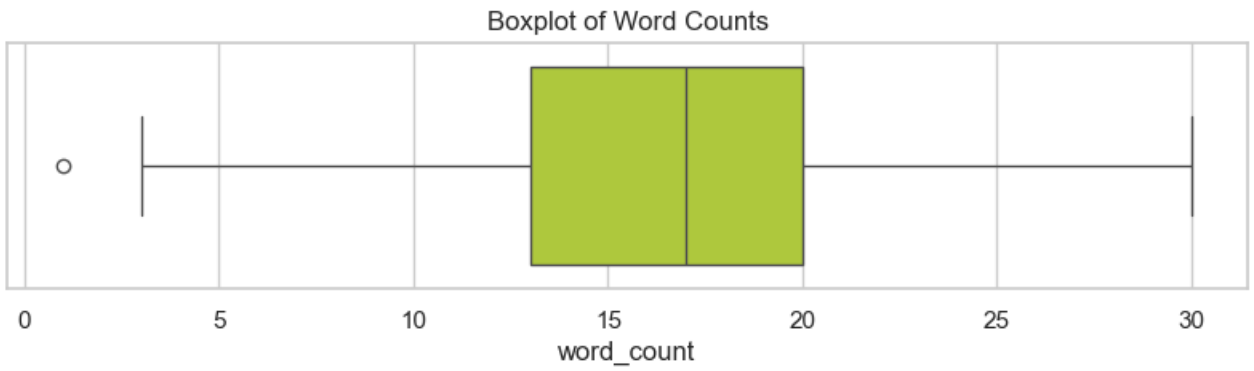
Out[34]:
```
count    9070.000000
mean       16.524807
std         4.768126
min         1.000000
25%        13.000000
50%        17.000000
75%        20.000000
max        30.000000
Name: word_count, dtype: float64
```

The average tweet contains approximately 17 words, with a median of 17 words, indicating a relatively balanced distribution. Most tweets contain between 13 and 20 words, while very short and very long tweets are uncommon. The maximum word count of 30 suggests that even the longest tweets remain relatively concise.

Word count boxplot

In [35]:
```
plt.figure(figsize=(10,2))

sns.boxplot(
    x=df['word_count'],
```

```
    color=plt.cm.viridis(0.9)
)

plt.title('Boxplot of Word Counts')

plt.show()
```



The boxplot shows that the majority of tweets contain between 13 and 20 words, with a median word count of approximately 17 words. The distribution appears relatively symmetrical, with limited variability and few extreme values. A small number of very short tweets are identified as outliers, while no substantial high-end outliers are observed.

Insights

The relatively narrow spread of word counts suggests that customers communicate their opinions in a consistent and concise manner. The absence of extreme outliers indicates that most tweets provide a similar amount of textual information, which can help improve the consistency and reliability of sentiment classification models.

4.2.4 Most Frequent Words Analysis

The purpose of this analysis is to identify the most commonly used words within the tweet dataset. Examining word frequencies helps uncover dominant discussion topics, recurring themes, and commonly referenced products or brands.

This analysis provides valuable context for understanding customer conversations and may reveal words that contribute significantly to sentiment classification.

Creating a Corpus of Words

The code below combines all tweets into a single text corpus and calculates the frequency of each word.

In [36]:
```
# Define stopwords
stop_words = set(stopwords.words('english'))

#project-specific stopwords
custom_stopwords = {
    'rt', 'amp', 'quot', 'im', 'dont', 'cant', 'youre',
    'apple', 'google'
}
```

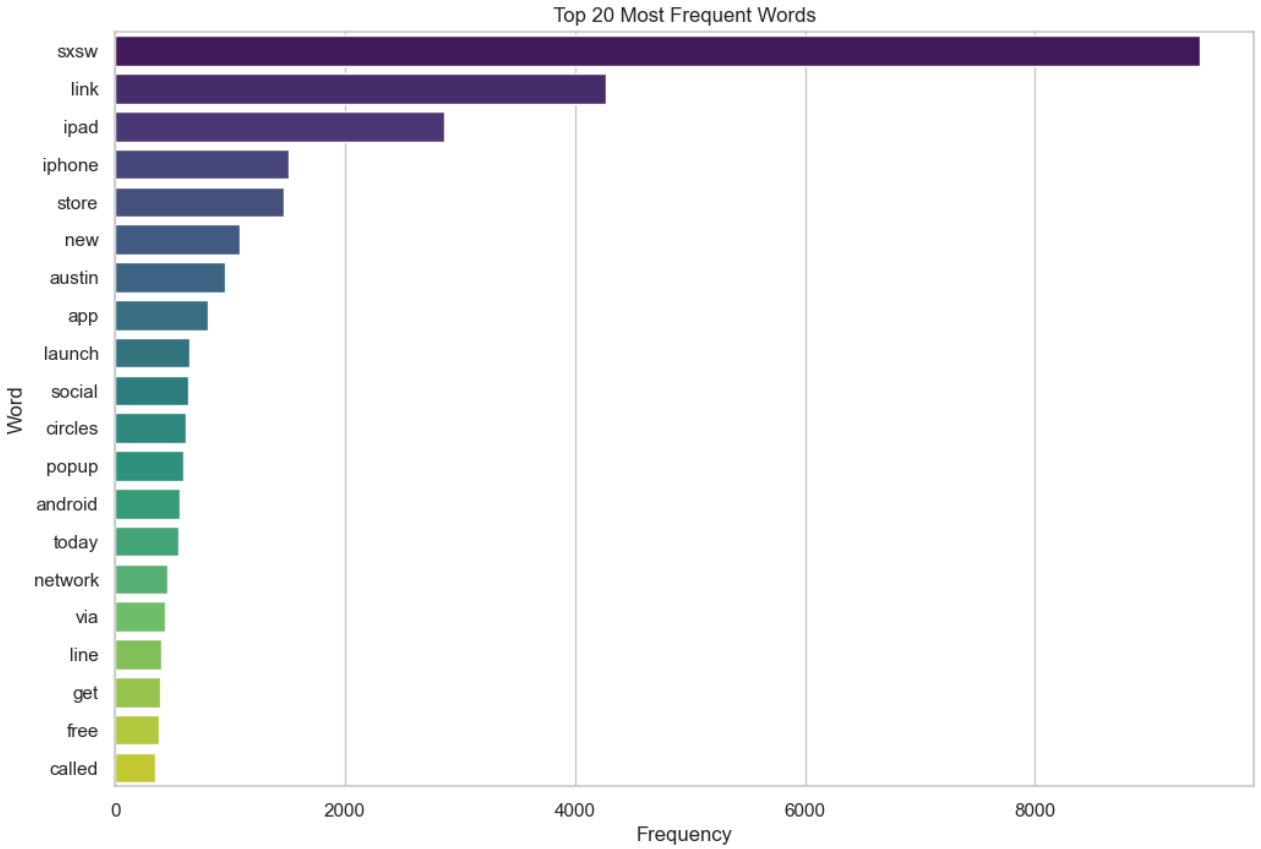
```
all_stopwords = stop_words.union(custom_stopwords)

# Create cleaned word list from tweet column
filtered_words = []

for tweet in df['tweet']:
    words = str(tweet).split()
    for word in words:
        word = word.lower().strip()
        if word not in all_stopwords and len(word) > 2:
            filtered_words.append(word)

# Count filtered words
word_freq = Counter(filtered_words)

# Create top words dataframe
top_words = pd.DataFrame(
    word_freq.most_common(20),
    columns=['Word', 'Frequency']
)

top_words
```

Out[36]:

	Word	Frequency
0	sxsw	9432
1	link	4268
2	ipad	2865
3	iphone	1515
4	store	1466
5	new	1082
6	austin	952
7	app	806
8	launch	643
9	social	634
10	circles	614
11	popup	599
12	android	562
13	today	556
14	network	456
15	via	435
16	line	401
17	get	393
18	free	383
19	called	354

Visualizing most frequent words

In [37]:
```
plt.figure(figsize=(12,8))

sns.barplot(
    data=top_words,
    x='Frequency',
    y='Word',
    hue='Word',
    palette='viridis',
    legend=False
)

plt.title('Top 20 Most Frequent Words')
plt.xlabel('Frequency')
plt.ylabel('Word')

plt.show()
```



Insights

The frequency analysis suggests that much of the conversation surrounding Apple and Google products occurred within the context of a major technology event. This highlights how industry conferences and product launches can significantly influence online discussions and customer engagement.

The prominence of product-specific terms such as iPad, iPhone, and Android indicates that these products generated substantial attention and may play an important role in shaping overall sentiment toward the brands.

The presence of event-specific terms such as SXSW highlights the importance of context when interpreting word frequencies. Not all commonly occurring words directly reflect customer sentiment; some capture the broader environment in which conversations took place.

4.2.5 Word Cloud Analysis

Word clouds help visualize the most prominent terms appearing in tweets and provide a quick overview of dominant discussion themes.

4.2.5.1 Overall Word Cloud

The purpose of the overall word cloud is to visualize the most frequently occurring words across all tweets. Larger words represent terms that appear more frequently within customer conversations.

The code below combines all tweets into a single text corpus and generates a word cloud using the Viridis color palette.

In [41]:
```
from wordcloud import WordCloud, STOPWORDS

custom_stopwords = {
    'sxsw', 'sxswi', 'link', 'rt', 'amp',
    'via', 'new', 'one', 'will', 'now',
    'today', 'get', 'going', 'called',
    'ûï', 'ûò', 'ûª', 'û', 'å'
}

all_stopwords = set(STOPWORDS).union(custom_stopwords)

# Create one text corpus
all_text = ' '.join(df['tweet'].astype(str))

# Remove non-letter characters and lowercase text
all_text_clean = re.sub(r'[^a-zA-Z\s]', ' ', all_text).lower()

# Split text into words and manually remove stopwords
filtered_words = [
    word for word in all_text_clean.split()
    if word not in all_stopwords and len(word) > 2
]

# Rejoin filtered words into clean text
filtered_text = ' '.join(filtered_words)

# Generate word cloud from filtered text
wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color='white',
    colormap='viridis',
    max_words=100
).generate(filtered_text)

plt.figure(figsize=(15,8))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
```

```
plt.title('Overall Word Cloud After Stopword Removal')
plt.show()
```

Insights

The overall word cloud highlights the most frequently discussed topics within customer conversations. Prominent terms such as Google, Apple, iPad, iPhone, App, and Store indicate that discussions were heavily centered around mobile devices, applications, and digital services.

The appearance of terms such as Launch, Social Network, Google Circle, and Popup Store suggests that many conversations focused on product announcements, technology events, and emerging digital platforms.

4.2.5.2 Positive Sentiment Word Cloud

This analysis helps to identify the words most frequently associated with positive sentiment. Understanding these terms helps reveal the features, products, and experiences that customers view favorably.

The code below filters tweets classified as Positive, removes common and project-specific stopwords, and generates a word cloud highlighting the most prominent positive terms.

In [42]:
```
# Extract positive tweets
positive_text = ' '.join(
    df[df['sentiment'] == 'Positive']['tweet'].astype(str)
)

# Additional stopwords
positive_stopwords = set(STOPWORDS).union({
    'google', 'apple', 'ipad', 'iphone',
    'app', 'apps', 'sxsw', 'sxswi',
    'link', 'rt', 'amp', 'via'
})
```

```
# Clean text
positive_text = re.sub(r'[^a-zA-Z\s]', ' ', positive_text).lower()

# Remove stopwords
filtered_positive = [
    word for word in positive_text.split()
    if word not in positive_stopwords and len(word) > 2
]

positive_text_clean = ' '.join(filtered_positive)

# Generate word cloud
positive_wc = WordCloud(
    width=1000,
    height=500,
    background_color='white',
    colormap='viridis',
    max_words=100
).generate(positive_text_clean)

# Plot
plt.figure(figsize=(15,8))
plt.imshow(positive_wc, interpolation='bilinear')
plt.axis('off')
plt.title('Positive Sentiment Word Cloud')
plt.show()
```

Insights

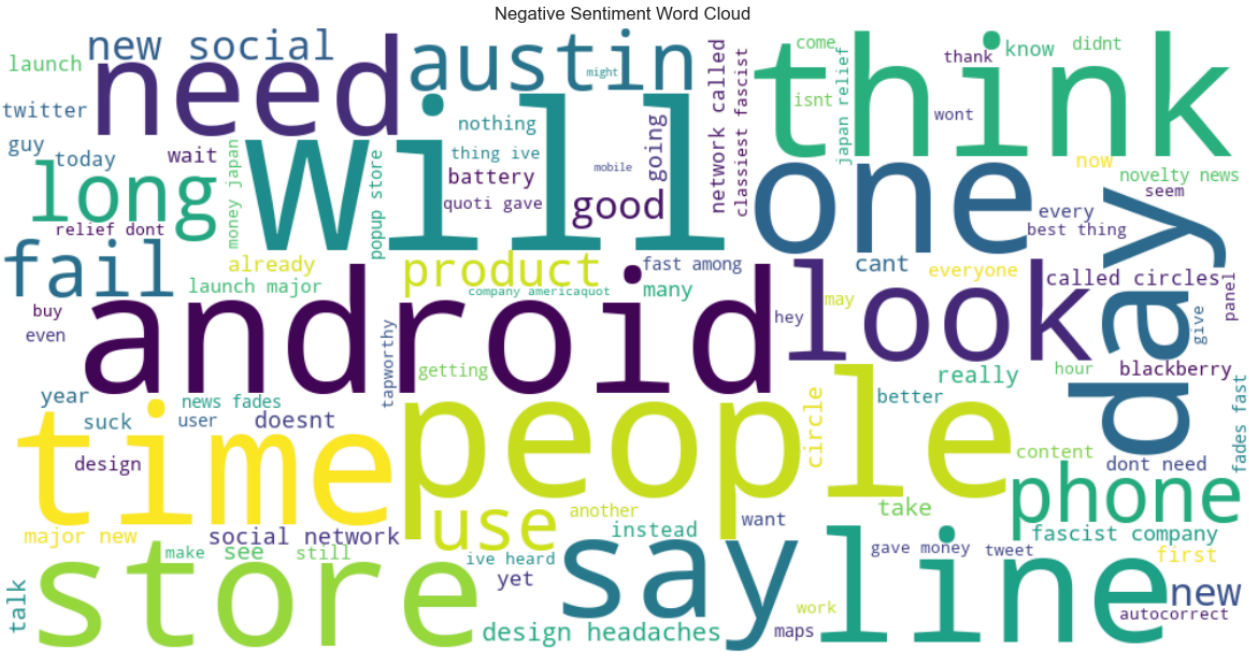
Prominent positive words such as great, awesome, good, love, nice, and fun suggest that many users expressed enthusiasm and satisfaction when discussing Apple and Google products. The prevalence of strongly positive terms suggests that customers generally viewed their experiences with Apple and Google products favorably. Discussions surrounding applications, stores, social networking features, and product-related events appear to be important drivers of positive engagement.

For stakeholders, this indicates that successful product launches, innovative features, and engaging user experiences may contribute significantly to positive public sentiment.

4.2.5.3 Negative Sentiment Word Cloud

This anallsis identifies the words most frequently associated with negative sentiment. Understanding these terms can help uncover customer frustrations, product concerns, and areas where improvements may be needed.

In [43]:
```
# Extract negative tweets

negative_text = ' '.join(
    df[df['sentiment'] == 'Negative']['tweet'].astype(str)
)

# Additional stopwords

negative_stopwords = set(STOPWORDS).union({
    'google', 'apple', 'ipad', 'iphone',
    'app', 'apps', 'sxsw', 'sxswi',
    'link', 'rt', 'amp', 'via'
})

# Clean text

negative_text = re.sub(r'[^a-zA-Z\s]', ' ', negative_text).lower()

# Remove stopwords

filtered_negative = [
    word for word in negative_text.split()
    if word not in negative_stopwords and len(word) > 2
]

negative_text_clean = ' '.join(filtered_negative)

# Generate word cloud

negative_wc = WordCloud(
    width=1000,
    height=500,
    background_color='white',
    colormap='viridis',
    max_words=100
).generate(negative_text_clean)

# Plot

plt.figure(figsize=(15,8))

plt.imshow(negative_wc, interpolation='bilinear')

plt.axis('off')

plt.title('Negative Sentiment Word Cloud')

plt.show()
```



Prominent words such as fail, headaches, battery, can't, doesn't, and fades suggest that users expressed concerns related to product performance, functionality, and user experience.The appearance of terms such as product, design, network, and social indicates that negative sentiment was often linked to product features, service reliability, and technology-related experiences.

Insights

The negative sentiment vocabulary suggests that customer dissatisfaction is primarily driven by usability issues, product performance concerns, and unmet expectations. Terms related to battery performance, technical failures, and functionality challenges highlight areas that may require further investigation by product and customer experience teams.

For stakeholders, monitoring these recurring concerns can help identify opportunities to improve product quality and customer satisfaction

4.3 Bivariate Analysis

Bivariate analysis explores relationships between two variables. This analysis helps uncover patterns that may influence customer sentiment and provides deeper insights into how product discussions and tweet characteristics relate to sentiment outcomes.

The findings from this section will help answer key business questions and guide feature engineering and model development.

4.3.1 Sentiment by Product

Understanding how sentiment varies across products helps identify which Apple and Google products generate the most favorable and unfavorable customer reactions.

The code below creates a cross-tabulation showing the distribution of sentiment categories across products.

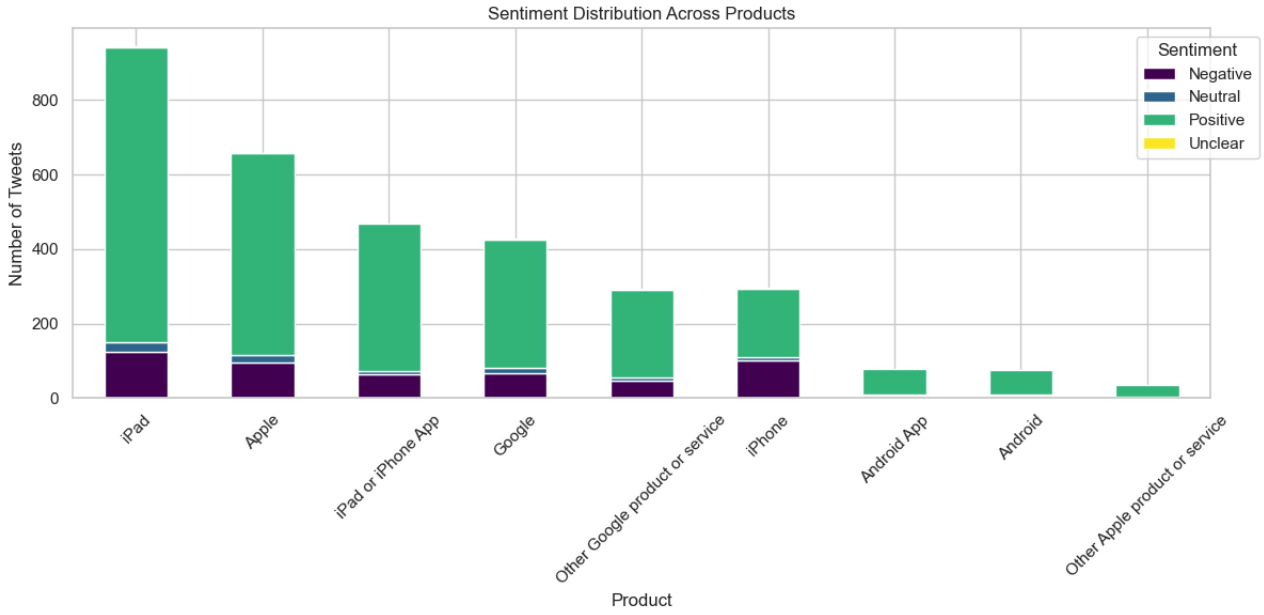
Sentiment Distribution Across Products

In [44]:
```
# Sentiment distribution by product

sentiment_product = pd.crosstab(
    df['product'],
    df['sentiment']
)

sentiment_product.head()
```

Out[44]:

sentiment product	Negative	Neutral	Positive	Unclear
Android	8	1	68	0
Android App	8	1	71	0
Apple	95	21	541	2
Google	68	15	344	1
Other Apple product or service	2	1	32	0

In [45]:
```
# Exclude Unknown products

filtered_df = df[df['product'] != 'Unknown']

sentiment_product = pd.crosstab(
    filtered_df['product'],
    filtered_df['sentiment']
)

sentiment_product.sort_values(
    by='Positive',
    ascending=False
).plot(
    kind='bar',
    stacked=True,
    figsize=(12,6),
    colormap='viridis'
)

plt.title('Sentiment Distribution Across Products')
plt.xlabel('Product')
plt.ylabel('Number of Tweets')

plt.xticks(rotation=45)

plt.legend(
    title='Sentiment',
    bbox_to_anchor=(1.05,1)
)

plt.tight_layout()

plt.show()
```



Insights

The visualization shows that positive sentiment is the dominant sentiment category across all products. The highest volume of sentiment is associated with iPad, followed by Apple, iPad or iPhone App, Google, and iPhone. While negative sentiment is present across several products, it represents a much smaller proportion of overall conversations.

Among the most frequently discussed products, the iPhone appears to attract a relatively larger share of negative sentiment compared to other products, suggesting potential differences in customer experiences or expectations.

4.3.2 Sentiment Composition by Product

While the previous analysis examined the volume of sentiment associated with each product, this analysis investigates the proportion of positive, negative, neutral, and unclear sentiment within each product category. By normalizing sentiment distributions, we can compare products fairly regardless of differences in discussion volume. We shall explore which products receive the highest proportion of positive and negative sentiment

Sentiment proportions

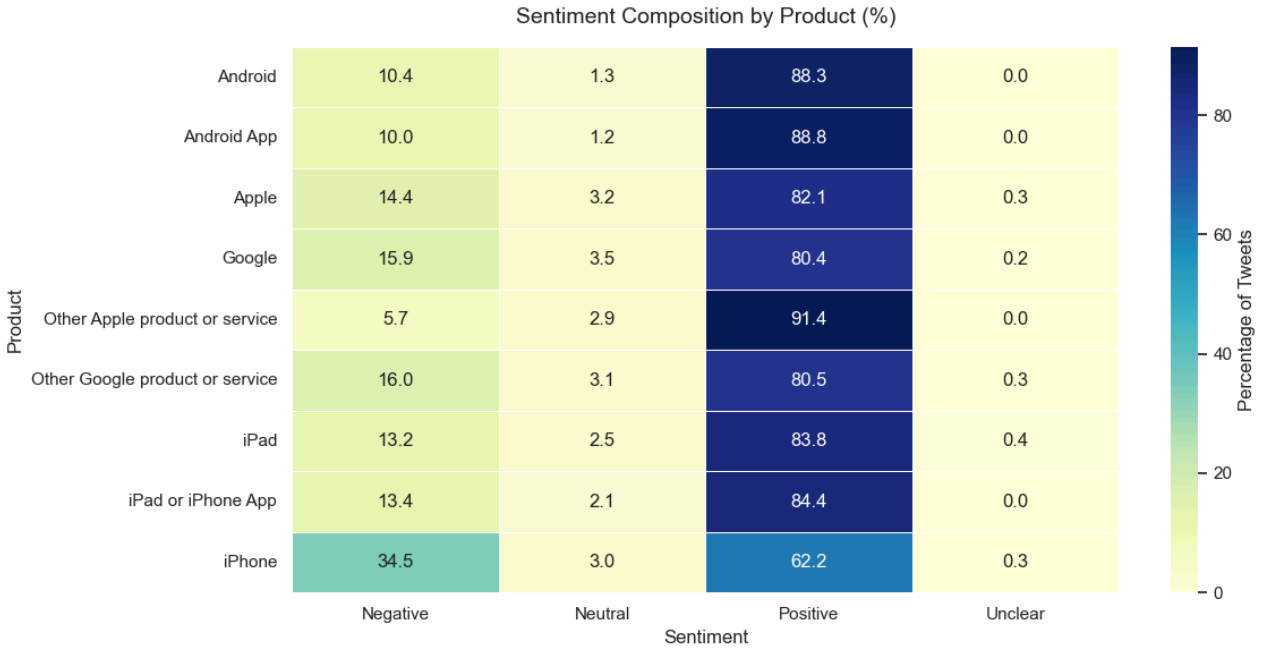
In [46]:
```
# Calculate sentiment percentages by product

sentiment_pct = pd.crosstab(
    filtered_df['product'],
    filtered_df['sentiment'],
    normalize='index'
) * 100

# Display results

sentiment_pct.round(1)
```

Out[46]:

sentiment	Negative	Neutral	Positive	Unclear
product				
Android	10.4	1.3	88.3	0.0
Android App	10.0	1.2	88.8	0.0
Apple	14.4	3.2	82.1	0.3
Google	15.9	3.5	80.4	0.2
Other Apple product or service	5.7	2.9	91.4	0.0
Other Google product or service	16.0	3.1	80.5	0.3
iPad	13.2	2.5	83.8	0.4
iPad or iPhone App	13.4	2.1	84.4	0.0
iPhone	34.5	3.0	62.2	0.3

In [47]:

```
plt.figure(figsize=(12,6))

sns.heatmap(
    sentiment_pct,
    annot=True,
    fmt='.1f',
    cmap='YlGnBu',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Percentage of Tweets'}
)

plt.title(
    'Sentiment Composition by Product (%)',
    fontsize=14,
    pad=15
)

plt.xlabel('Sentiment')
plt.ylabel('Product')

plt.tight_layout()
plt.show()
```



Insights The heatmap reveals that most Apple and Google products enjoy overwhelmingly positive customer sentiment, suggesting strong brand perception and generally favorable user experiences. Products such as Android, Android App, and iPad or iPhone App demonstrate particularly high positivity rates, indicating that customers respond well to these offerings.

However, the iPhone emerges as a notable exception, exhibiting the highest negative sentiment rate (34.2%) and the lowest positive sentiment rate (62.4%) among all products. This suggests that iPhone users may have higher expectations, encounter more product-related frustrations, or be more vocal about issues compared to users of other products.

For stakeholders, this finding highlights the importance of monitoring customer feedback at the product level rather than relying solely on overall brand sentiment. While overall sentiment toward Apple and Google products is largely positive, product-specific challenges may be hidden within aggregate sentiment metrics.

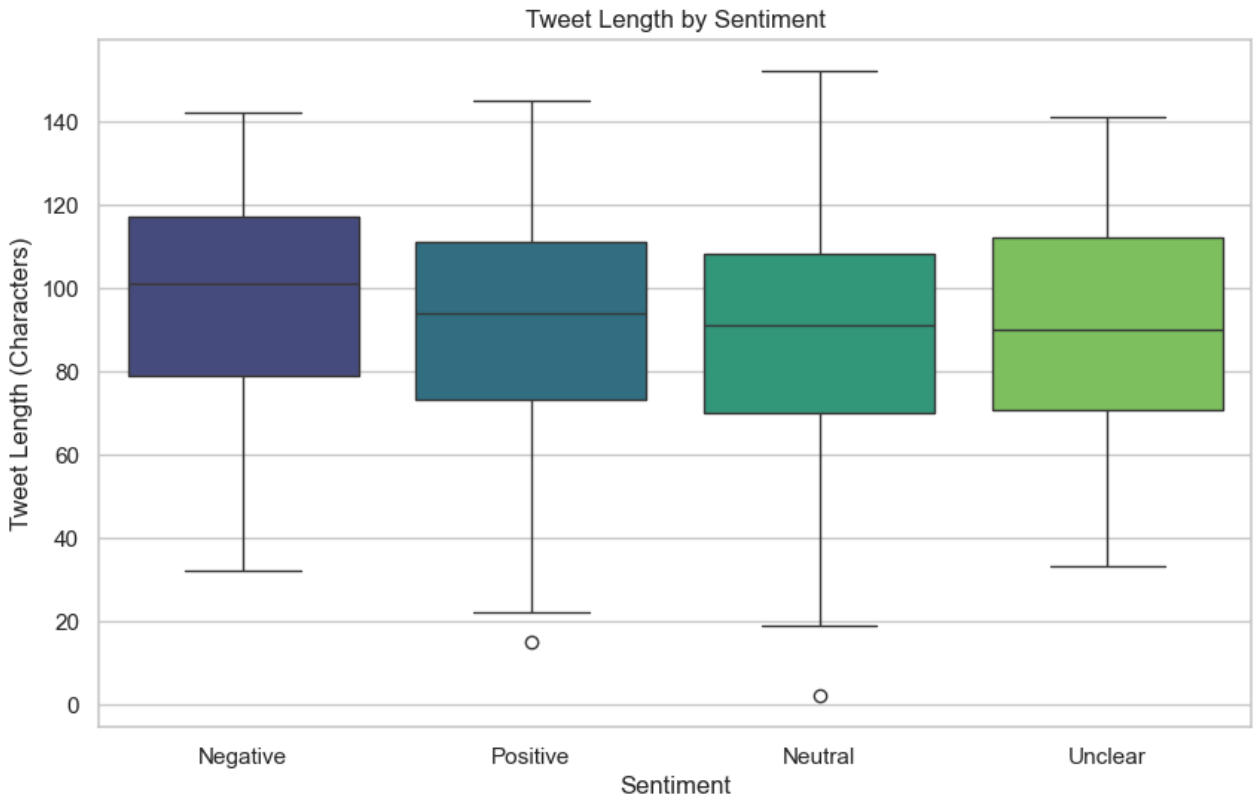
4.3.3 Tweet Length by Sentiment

This analysis investigates whether tweet length varies across sentiment categories. Understanding these differences may provide insights into how customers communicate positive, negative, neutral, and unclear opinions.

Tweet Length By Sentiment

In [48]:
```
plt.figure(figsize=(10,6))

sns.boxplot(
    data=df,
    x='sentiment',
    y='tweet_length',
    hue='sentiment',
    palette='viridis',
    legend=False
)
```

```
plt.title('Tweet Length by Sentiment')
plt.xlabel('Sentiment')
plt.ylabel('Tweet Length (Characters)')

plt.show()
```



Insights

The boxplot shows that tweet lengths are generally similar across all sentiment categories, with median lengths ranging between approximately 90 and 100 characters. Negative tweets have a slightly higher median length, suggesting that users expressing dissatisfaction may provide more detailed feedback. Overall, the differences are relatively small, indicating that tweet length alone is unlikely to strongly differentiate sentiment categories.

The limited variation in tweet length across sentiment categories suggests that features derived from the actual text content (such as word frequencies, TF-IDF scores, or n-grams) are likely to be more valuable for sentiment classification than simple length-based features.

4.3.4 Word Count by Sentiment

The purpose of this analysis is to examine whether the number of words used in a tweet varies across sentiment categories. These differences can provide insight into how customers communicate positive, negative, and neutral opinions.

Summary Statistics

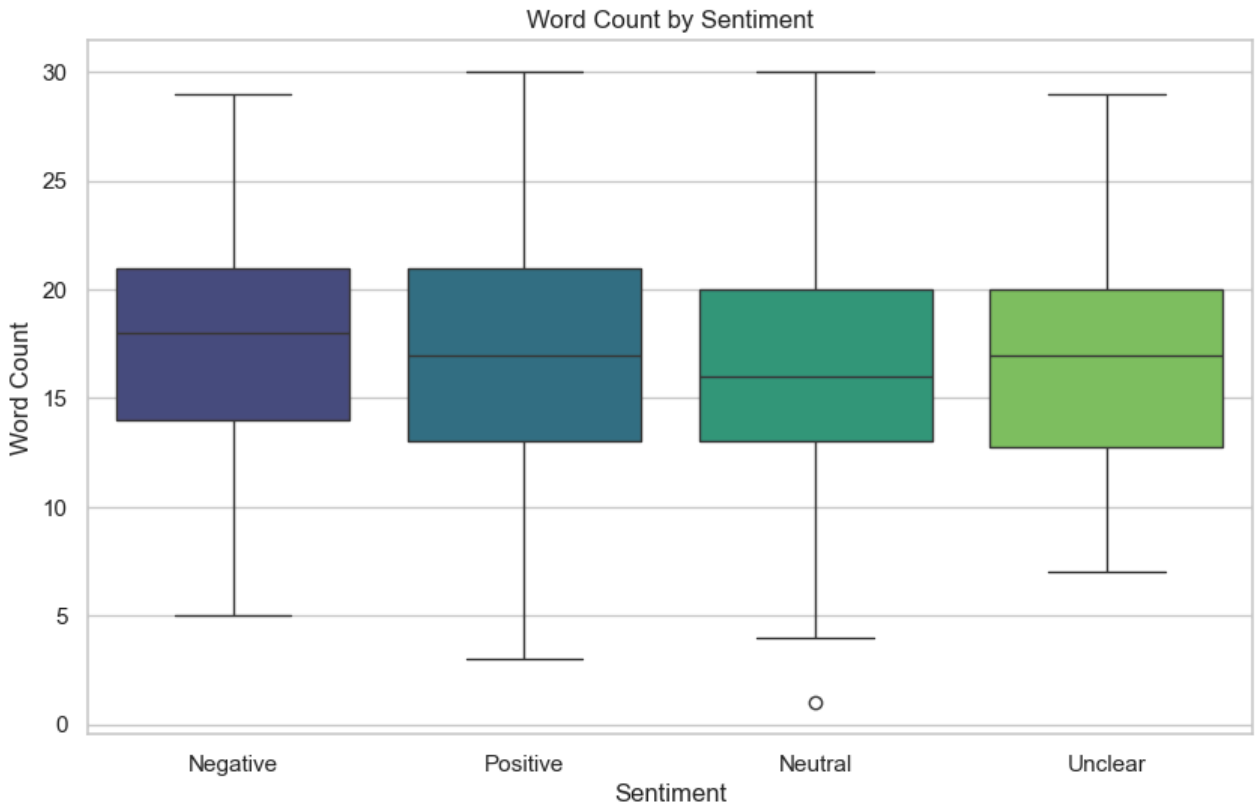
In [49]:
```
df.groupby('sentiment')['word_count'].describe()
```

Out[49]:

sentiment	count	mean	std	min	25%	50%	75%	max
Negative	569.0	17.797891	4.933672	5.0	14.00	18.0	21.0	29.0
Neutral	5375.0	16.146791	4.704660	1.0	13.00	16.0	20.0	30.0
Positive	2970.0	16.967003	4.768143	3.0	13.00	17.0	21.0	30.0
Unclear	156.0	16.487179	4.845333	7.0	12.75	17.0	20.0	29.0

The code below compares word count distributions across sentiment categories using a boxplot.

In [50]:
```
plt.figure(figsize=(10,6))

sns.boxplot(
    data=df,
    x='sentiment',
    y='word_count',
    hue='sentiment',
    palette='viridis',
    legend=False
)

plt.title('Word Count by Sentiment')
plt.xlabel('Sentiment')
plt.ylabel('Word Count')

plt.show()
```



Insights

The boxplot shows that word counts are relatively consistent across all sentiment categories, with median word counts ranging from approximately 16 to 18 words. Negative tweets have a slightly higher median word count, while neutral tweets tend to contain slightly fewer words. Overall, the differences are small, suggesting that sentiment is not strongly associated with the number of words used in a tweet. This suggests that the content of the message is likely more important than the quantity of words when determining sentiment.

5. Data Preparation

This section prepares the final modeling dataset using NLP preprocessing, feature selection, and vectorization techniques.cleaned tweet text is transformed into a format that machine learning models can understand. Since the models cannot learn directly from raw text, the tweets must be processed, converted into numerical features, and split into training and testing datasets.

5.1 Feature and Target Selection

Definition of the input feature and the target variable.

In [129…

```
# Define feature and target variables

X = df['tweet']
y = df['sentiment']
```

Tweet text is the most relevant predictor because it contains the actual customer language, opinions, and emotional signals needed for sentiment classification.

5.2 NLP Text Preprocessing

This step focuses on reducing text noise and improving consistency so that models can learn meaningful language patterns.

5.2.1 Convert Text to Lowercase

Converting text to lowercase ensures consistency across all tweets.

In [105…

```
df['processed_tweet'] = df['tweet'].str.lower()

df[['tweet', 'processed_tweet']].head()
```

Out[105…

	tweet	processed_tweet
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw
2	can not wait for ipad also they should sale them down at sxsw	can not wait for ipad also they should sale them down at sxsw

	tweet	processed_tweet
3	i hope this years festival isnt as crashy as this years iphone app sxsw	i hope this years festival isnt as crashy as this years iphone app sxsw
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress

5.2.2 Remove Non-Letter Characters

Tweets often contain numbers, punctuation, special symbols, hashtags, and other characters that may not contribute meaningfully to sentiment prediction.

In [106…
```
df['processed_tweet'] = df['processed_tweet'].apply(
    lambda text: re.sub(r'\s+', ' ', str(text)).strip()
)

df[['tweet', 'processed_tweet']].head()
```

Out[106…

	tweet	processed_tweet
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw
2	can not wait for ipad also they should sale them down at sxsw	can not wait for ipad also they should sale them down at sxsw
3	i hope this years festival isnt as crashy as this years iphone app sxsw	i hope this years festival isnt as crashy as this years iphone app sxsw
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress

5.2.3 Remove Extra Whitespace

Text cleaning may introduce multiple spaces between words. These unnecessary spaces can affect downstream text processing and should be standardized.

In [107…
```
df['processed_tweet'] = df['processed_tweet'].apply(
    lambda text: re.sub(r'\s+', ' ', str(text)).strip()
)

df[['tweet', 'processed_tweet']].head()
```

Out[107…

	tweet	processed_tweet
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at

	tweet	processed_tweet
	sxsw	sxsw
2	can not wait for ipad also they should sale them down at sxsw	can not wait for ipad also they should sale them down at sxsw
3	i hope this years festival isnt as crashy as this years iphone app sxsw	i hope this years festival isnt as crashy as this years iphone app sxsw
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress

5.3 Stopword Removal

Removing English stopwords from the tweet text.

In [108…

```
stop_words = set(stopwords.words('english'))

df['processed_tweet'] = df['processed_tweet'].apply(
    lambda text: ' '.join(
        word for word in text.split()
        if word not in stop_words
    )
)

df[['tweet', 'processed_tweet']].head()
```

Out[108…

	tweet	processed_tweet
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	g iphone hrs tweeting riseaustin dead need upgrade plugin stations sxsw
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	know awesome ipadiphone app youll likely appreciate design also theyre giving free ts sxsw
2	can not wait for ipad also they should sale them down at sxsw	wait ipad also sale sxsw
3	i hope this years festival isnt as crashy as this years iphone app sxsw	hope years festival isnt crashy years iphone app sxsw
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	great stuff fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress

5.4 Lemmatization

Converting words to their root forms using WordNetLemmatizer.

In [109…

```
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

df['processed_tweet'] = df['processed_tweet'].apply(
```

```
    lambda text: ' '.join(
        lemmatizer.lemmatize(word)
        for word in text.split()
    )
)

df[['tweet', 'processed_tweet']].head()
```

Out[109…

	tweet	processed_tweet
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	g iphone hr tweeting riseaustin dead need upgrade plugin station sxsw
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	know awesome ipadiphone app youll likely appreciate design also theyre giving free t sxsw
2	can not wait for ipad also they should sale them down at sxsw	wait ipad also sale sxsw
3	i hope this years festival isnt as crashy as this years iphone app sxsw	hope year festival isnt crashy year iphone app sxsw
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	great stuff fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress

5.5 Feature Engineering

Feature engineering involves creating additional variables that may provide useful information to machine learning models. While sentiment is primarily driven by the text content, structural characteristics such as tweet length and word count may also contain predictive information.

Based on the EDA findings, tweet length and word count showed limited variation across sentiment categories. However, these features will be retained and evaluated during the modeling process.

5.5.1 Tweet Length Feature

Tweet length measures the total number of characters in a tweet and may provide insight into how users communicate different sentiments.

In [110…

```
# Create tweet length feature

df['tweet_length'] = df['tweet'].apply(len)

# Preview results

df[['tweet', 'tweet_length']].head()
```

Out[110…

	tweet	tweet_length
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	104
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	112

	tweet	tweet_length
2	can not wait for ipad also they should sale them down at sxsw	61
3	i hope this years festival isnt as crashy as this years iphone app sxsw	71
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	107

A new feature representing the number of characters in each tweet has been created.

5.5.2 Word Count Feature

Word count measures the number of words contained within each tweet and may help capture communication patterns.

In [111…

```
# word count feature

df['word_count'] = df['tweet'].apply(
    lambda text: len(str(text).split())
)

# Preview results

df[['tweet', 'word_count']].head()
```

Out[111…

	tweet	word_count
0	i have a g iphone after hrs tweeting at riseaustin it was dead i need to upgrade plugin stations at sxsw	21
1	know about awesome ipadiphone app that youll likely appreciate for its design also theyre giving free ts at sxsw	19
2	can not wait for ipad also they should sale them down at sxsw	13
3	i hope this years festival isnt as crashy as this years iphone app sxsw	14
4	great stuff on fri sxsw marissa mayer google tim oreilly tech booksconferences amp matt mullenweg wordpress	16

A new feature representing the number of words in each tweet has been created.

Summary Statistics for Engineered Features

In [112…

```
# summary statistics for engineered features

df[['tweet_length', 'word_count']].describe()
```

Out[112…

	tweet_length	word_count
count	9070.000000	9070.000000
mean	89.957332	16.524807
std	25.133610	4.768126
min	2.000000	1.000000
25%	72.000000	13.000000

	tweet_length	word_count
50%	92.000000	17.000000
75%	109.000000	20.000000
max	152.000000	30.000000

5.6 Train-Test Split

The dataset was split into training and testing sets using an 80/20 ratio. Stratified sampling was applied to ensure that the distribution of sentiment classes remained consistent across both datasets.

In [113…

```
# Import libraries
from sklearn.model_selection import train_test_split

# Define the predictor variable
# The model will use the processed tweet text to predict sentiment
X = df['processed_tweet']

# Define the target variable
# The target contains the sentiment labels
y = df['sentiment']

# Split the data into training and testing sets
# test_size=0.20 reserves 20% of the data for final evaluation
# random_state=42 ensures reproducibility
# stratify=y preserves the sentiment distribution in both datasets
X_train, X_test, y_train, y_test = train_test_split(X,  y, test_size=0.20,random_state=

# Display the number of observations in each dataset
print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

# Verify that class proportions were preserved after splitting
print("\nTraining Set Distribution (%)")
print((y_train.value_counts(normalize=True) * 100).round(2))

print("\nTesting Set Distribution (%)")
print((y_test.value_counts(normalize=True) * 100).round(2))
```

```
Training set size: (7256,)
Testing set size: (1814,)

Training Set Distribution (%)
sentiment
Neutral     59.26
Positive    32.75
Negative     6.27
Unclear      1.72
Name: proportion, dtype: float64

Testing Set Distribution (%)
sentiment
Neutral     59.26
Positive    32.75
Negative     6.28
```

```
Unclear      1.71
Name: proportion, dtype: float64
```

5.7 Validation Strategy

To ensure that model performance is evaluated fairly and generalizes well to unseen data.

The dataset was split into training (80%) and testing (20%) sets using stratified sampling to preserve the original sentiment distribution.During model development, 5-fold stratified cross-validation will be performed on the training data. This approach provides a more reliable estimate of model performance and helps reduce the risk of overfitting.

The holdout test set will remain untouched throughout model training and selection and will only be used for the final evaluation of the best-performing model.

Define Cross-Validation Strategy

In [114…

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

print(cv)
```

```
StratifiedKFold(n_splits=5, random_state=42, shuffle=True)
```

This cross-validation strategy will be applied during model training and comparison in the modeling section.

6. Text Vectorization

Tweet texts will be converted into numerical features that machine learning algorithms can understand and process.

6.1 CountVectorizer

CountVectorizer converts tweets into numerical features based on word frequencies.Each tweet is transformed into a vector where each feature represents the number of times a word appears. CountVectorizer serves as a simple and interpretable baseline representation of text and will be used with Logistic Regression to establish an initial NLP benchmark.

In [115…

```
# Import CountVectorizer
from sklearn.feature_extraction.text import CountVectorizer

# Initialize CountVectorizer
count_vectorizer = CountVectorizer()

# Fit CountVectorizer on training data
# Transform both training and testing datasets
```

```
X_train_count = count_vectorizer.fit_transform(X_train)

X_test_count = count_vectorizer.transform(X_test)

# Display the dimensions of the CountVectorizer feature matrices

print("Training Matrix Shape:", X_train_count.shape)
print("Testing Matrix Shape:", X_test_count.shape)
```

```
Training Matrix Shape: (7256, 8509)
Testing Matrix Shape: (1814, 8509)
```

CountVectorizer has converted the tweet text into a numerical feature matrix suitable for machine learning.

6.2 TF-IDF Vectorization

While CountVectorizer treats all words equally, TF-IDF assigns greater importance to words that are distinctive and potentially more informative for sentiment classification. Common words appearing in many tweets contribute less predictive value than words that are unique to specific sentiment categories.

In [116…

```
# Import TF-IDF Vectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer()

# Fit TF-IDF on training data
# Transform both training and testing datasets

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)

X_test_tfidf = tfidf_vectorizer.transform(X_test)

# Display the dimensions of the TF-IDF feature matrices

print("Training Matrix Shape:", X_train_tfidf.shape)
print("Testing Matrix Shape:", X_test_tfidf.shape)
```

```
Training Matrix Shape: (7256, 8509)
Testing Matrix Shape: (1814, 8509)
```

TF-IDF has transformed the tweets into weighted numerical features that emphasize important and distinctive words.

7. Modeling

In this section we train and compare multiple supervised machine learning models for sentiment classification. Each model will be evaluated using stratified cross-validation on the training data. The holdout test set will remain untouched until the final evaluation section.

7.1 Model Evaluation Framework

To compare models fairly, each model will be evaluated using the same validation approach and performance metrics. Since the sentiment classes are imbalanced, weighted F1-score will be used as the primary metric for model selection.

In [117…

```
# Define a function to evaluate models using cross-validation

def evaluate_model(model, X_train_data, y_train_data, model_name):
    """
    Evaluates a machine learning model using stratified cross-validation.

    """

    # Define scoring metrics
    scoring = {
        'accuracy': 'accuracy',
        'precision': 'precision_weighted',
        'recall': 'recall_weighted',
        'f1': 'f1_weighted'
    }

    # Run cross-validation on the training data
    cv_results = cross_validate(
        model,
        X_train_data,
        y_train_data,
        cv=cv,
        scoring=scoring,
        n_jobs=-1
    )

    # Store average cross-validation results
    results = {
        'Model': model_name,
        'CV Accuracy': cv_results['test_accuracy'].mean(),
        'CV Precision': cv_results['test_precision'].mean(),
        'CV Recall': cv_results['test_recall'].mean(),
        'CV F1': cv_results['test_f1'].mean()
    }

    return results
```

This function allows all models to be evaluated consistently using the same cross-validation strategy and metrics.

7.2 Baseline Model: Dummy Classifier

Before training machine learning models, a baseline benchmark is established using a Dummy Classifier. The Dummy Classifier does not learn from tweet content and instead predicts the most frequent sentiment class.This provides a minimum performance threshold that all subsequent models should exceed.

Train and Evaluate the Dummy Classifier

In [118…

```
# Create a Dummy Classifier that predicts the most frequent class
dummy_model = DummyClassifier(
```

```
    strategy='most_frequent'
)

# Evaluate the baseline model using cross-validation
dummy_results = evaluate_model(
    model=dummy_model,
    X_train_data=X_train_tfidf,
    y_train_data=y_train,
    model_name='Dummy Classifier'
)
#Results
dummy_results
```

Out[118…

```
{'Model': 'Dummy Classifier',
 'CV Accuracy': np.float64(0.5926130549274462),
 'CV Precision': np.float64(0.3511902595297457),
 'CV Recall': np.float64(0.5926130549274462),
 'CV F1': np.float64(0.44102394149804935)}
```

The Dummy Classifier achieved a cross-validation accuracy of 59.3% and a weighted F1-score of 44.1%. Since the model predicts only the most frequent sentiment class and does not learn from tweet content, these results represent the minimum performance benchmark for the project.

The relatively high accuracy reflects the class imbalance observed during EDA, where certain sentiment categories occurred much more frequently than others.

7.3 Model 1: CountVectorizer + Logistic Regression

This model combines CountVectorizer and Logistic Regression to classify tweet sentiment. CountVectorizer transforms each tweet into a numerical representation based on the frequency of words appearing in the text, while Logistic Regression learns the relationship between these word frequencies and the corresponding sentiment labels.

Train and Evaluate CountVectorizer + Logistic Regression

In [119…

```
# Create Logistic Regression model
count_lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# Evaluate the model using CountVectorizer features
count_lr_results = evaluate_model(
    model=count_lr_model,
    X_train_data=X_train_count,
    y_train_data=y_train,
    model_name='CountVectorizer + Logistic Regression'
)

# Results
count_lr_results
```

Out[119…

```
{'Model': 'CountVectorizer + Logistic Regression',
 'CV Accuracy': np.float64(0.6783383930147918),
 'CV Precision': np.float64(0.6564928146113216),
 'CV Recall': np.float64(0.6783383930147918),
 'CV F1': np.float64(0.6612997462230572)}
```

The CountVectorizer + Logistic Regression model achieved a cross-validation accuracy of 67.9% and a weighted F1-score of 66.2%. Compared to the Dummy Classifier, the model shows a substantial improvement across all evaluation metrics, indicating that it successfully learns sentiment-related patterns from tweet text.

This improvement suggests that word frequency information provides valuable signals for distinguishing between sentiment categories.

7.4 Model 2: TF-IDF + Logistic Regression

This model combines TF-IDF Vectorization and Logistic Regression for sentiment classification. TF-IDF (Term Frequency-Inverse Document Frequency) transforms tweet text into weighted numerical features that reflect both how frequently a word appears in a tweet and how unique that word is across the dataset.

By reducing the influence of common terms and emphasizing more informative words, TF-IDF often produces stronger predictive performance than simple word frequency representations.

Train and Evaluate TF-IDF + Logistic Regression

In [120…
```
# Create Logistic Regression model
tfidf_lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# Evaluate the model using TF-IDF features
tfidf_lr_results = evaluate_model(
    model=tfidf_lr_model,
    X_train_data=X_train_tfidf,
    y_train_data=y_train,
    model_name='TF-IDF + Logistic Regression'
)

# Results
tfidf_lr_results
```

Out[120…
```
{'Model': 'TF-IDF + Logistic Regression',
 'CV Accuracy': np.float64(0.6736510205747722),
 'CV Precision': np.float64(0.6557287557611022),
 'CV Recall': np.float64(0.6736510205747722),
 'CV F1': np.float64(0.6341529930979446)}
```

The TF-IDF + Logistic Regression model achieved a cross-validation accuracy of 67.4% and a weighted F1-score of 63.4%. While the model substantially outperformed the Dummy Classifier, it performed slightly worse than the CountVectorizer + Logistic Regression model across all evaluation metrics.

This suggests that weighting words by importance did not improve sentiment classification performance for this dataset. The results indicate that simple word frequency information may be sufficient for capturing sentiment patterns in these tweets. Since sentiment-bearing words appear frequently and consistently across the dataset, reducing the influence of common terms through TF-IDF may have removed information that was useful for classification.

7.5 Model Optimization: Logistic Regression Hyperparameter Tuning

The objective of this step is to improve the performance of the TF-IDF + Logistic Regression model by identifying the optimal hyperparameter settings. Hyperparameter tuning helps determine whether the model can achieve better generalization and predictive performance than the default configuration.

In [121…

```
# Import GridSearchCV
from sklearn.model_selection import GridSearchCV

# Define the hyperparameter search space
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100]
}

# Configure Grid Search
grid_search = GridSearchCV(
    estimator=LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    param_grid=param_grid,
    cv=cv,
    scoring='f1_weighted',
    n_jobs=-1
)

# Perform hyperparameter tuning on the TF-IDF training data
grid_search.fit(
    X_train_tfidf,
    y_train
)

# Retrieve the best-performing model
tuned_lr_model = grid_search.best_estimator_

# Display the optimal hyperparameter configuration
print("Best Parameters:", grid_search.best_params_)

# Display the best cross-validation score
print(
    "Best Weighted F1 Score:",
    round(grid_search.best_score_, 4)
)
```

```
Best Parameters: {'C': 10}
Best Weighted F1 Score: 0.6606
```

The Grid Search identified C = 10 as the optimal regularization parameter for Logistic Regression, achieving a best cross-validation weighted F1-score of 0.6609.Compared to the untuned TF-IDF + Logistic Regression model (F1 = 0.634), tuning produced a meaningful improvement. However, the tuned model still slightly underperformed the CountVectorizer + Logistic Regression model (F1 = 0.662).

the results also suggest that the choice of text representation had a greater impact on performance than tuning alone. This reinforces the importance of evaluating both feature engineering and model optimization when developing sentiment classification systems.

7.6 Model 3: TF-IDF + LinearSVC

This evaluates whether a Linear Support Vector Classifier can improve sentiment classification performance using TF-IDF features. LinearSVC is commonly used for high-dimensional text classification problems and often performs well with sparse text data.

Train and Evaluate TF-IDF + LinearSVC

In [122…
```
# Create LinearSVC model
svc_model = LinearSVC(
    random_state=42
)

# Evaluate LinearSVC using TF-IDF features
svc_results = evaluate_model(
    model=svc_model,
    X_train_data=X_train_tfidf,
    y_train_data=y_train,
    model_name='TF-IDF + LinearSVC'
)

# Results
svc_results
```

Out[122…
```
{'Model': 'TF-IDF + LinearSVC',
 'CV Accuracy': np.float64(0.6784767036317692),
 'CV Precision': np.float64(0.6585568501908561),
 'CV Recall': np.float64(0.6784767036317692),
 'CV F1': np.float64(0.6603990066685149)}
```

The TF-IDF + LinearSVC model achieved a cross-validation accuracy of 67.8% and a weighted F1-score of 66.0%. The model substantially outperformed the Dummy Classifier and performed similarly to the tuned TF-IDF + Logistic Regression model.

Although LinearSVC demonstrated strong predictive performance, it did not surpass the CountVectorizer + Logistic Regression model, which remains the highest-performing model based on weighted F1-score.

The results suggest that both Logistic Regression and LinearSVC are effective classifiers for sentiment analysis. However, the relatively small performance differences between the top-performing models indicate that the choice of text representation may be as important as the choice of classification algorithm.

7.7 Advanced Model: TF-IDF + Random Forest

In this project, Random Forest was evaluated as an advanced machine learning model to determine whether an ensemble approach could improve sentiment classification performance compared to the previously evaluated models.

Train and Evaluate TF-IDF + Random Forest

In [131…
```
# Import Random Forest Classifier
from sklearn.ensemble import RandomForestClassifier

# Create Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=30,
    random_state=42,
    n_jobs=-1,
    class_weight='balanced'
)

# Evaluate Random Forest using TF-IDF features
rf_results = evaluate_model(
    model=rf_model,
    X_train_data=X_train_tfidf,
    y_train_data=y_train,
    model_name='TF-IDF + Random Forest'
)

# Display results
rf_results
```

Out[131…
```
{'Model': 'TF-IDF + Random Forest',
 'CV Accuracy': np.float64(0.5565090476217598),
 'CV Precision': np.float64(0.6174149897699774),
 'CV Recall': np.float64(0.5565090476217598),
 'CV F1': np.float64(0.5774689503157666)}
```

7.8 Model Comparison and Selection

A model comparison table is created to compare and select the winning model.

Model Comparison Table

In [ ]:
```
# Create model comparison table

model_results = pd.DataFrame([
    dummy_results,
    count_lr_results,
    tfidf_lr_results,
    {
        'Model': 'Tuned TF-IDF + Logistic Regression',
        'CV Accuracy': None,
        'CV Precision': None,
        'CV Recall': None,
        'CV F1': 0.6609
    },
    svc_results,
    rf_results
])

model_results.sort_values(
    by='CV F1',
    ascending=False
)
```

Out[ ]:

	Model	CV Accuracy	CV Precision	CV Recall	CV F1
1	CountVectorizer + Logistic Regression	0.678338	0.656493	0.678338	0.661300
3	Tuned TF-IDF + Logistic Regression	NaN	NaN	NaN	0.660900
4	TF-IDF + LinearSVC	0.678477	0.658557	0.678477	0.660399
2	TF-IDF + Logistic Regression	0.673651	0.655729	0.673651	0.634153
5	TF-IDF + Random Forest	0.556509	0.617415	0.556509	0.577469
0	Dummy Classifier	0.592613	0.351190	0.592613	0.441024

The model comparison results show that all machine learning models substantially outperformed the Dummy Classifier, confirming that tweet text contains meaningful information for sentiment classification. Among the evaluated models, **CountVectorizer + Logistic Regression** achieved the highest weighted F1-score (0.662) and the highest overall accuracy (67.9%). The tuned TF-IDF + Logistic Regression and TF-IDF + LinearSVC models produced competitive results, but neither exceeded the performance of the CountVectorizer-based model.

An advanced ensemble model, **TF-IDF + Random Forest**, was also evaluated. Despite its increased complexity, it achieved the lowest performance among the machine learning models, indicating that the high-dimensional and sparse nature of the text data was more effectively handled by the linear models.

Interestingly, the TF-IDF representation did not improve performance over simple word frequency counts, suggesting that raw word occurrence patterns were more informative for this dataset.

Final Model Selection

Based on cross-validation performance, CountVectorizer + Logistic Regression was selected as the final model because it:

(i) Achieved the highest weighted F1-score.

(ii) Maintained strong accuracy, precision, and recall.

(iii) Outperformed both TF-IDF-based models and the advanced ensemble model.

(iv) Provided greater interpretability than more complex alternatives.

(v) Offered a strong balance between predictive performance, computational efficiency, and model transparency.

8. Model Evaluation

8.1 Model Performance Evaluation

The performance of the selected model CountVectorizer + Logistic Regression is evaluated on the holdout test set. This provides an unbiased assessment of how well the model generalizes to unseen

data and its suitability for real-world sentiment classification.

Train, Predict, and Evaluate the Final Model

In [124…

```
# Create the final Logistic Regression model
final_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# Train the model using CountVectorizer features
final_model.fit(X_train_count,y_train)

# Generate predictions on the holdout test set
y_pred = final_model.predict(X_test_count)

# Calculate performance metrics
test_accuracy = accuracy_score(y_test,y_pred)

test_precision = precision_score(y_test, y_pred, average='weighted')

test_recall = recall_score( y_test, y_pred, average='weighted')

test_f1 = f1_score(y_test,y_pred,average='weighted')

# Results
print(f"Accuracy: {test_accuracy:.4f}")
print(f"Precision: {test_precision:.4f}")
print(f"Recall: {test_recall:.4f}")
print(f"F1 Score: {test_f1:.4f}")
```

```
Accuracy: 0.6670
Precision: 0.6464
Recall: 0.6670
F1 Score: 0.6504
```

The final CountVectorizer + Logistic Regression model achieved an accuracy of 66.7% and a weighted F1-score of 65.0% on the holdout test set. The results are broadly consistent with the cross-validation scores obtained during model development, indicating that the model generalizes well to unseen data.

The model demonstrates a reasonable ability to classify customer sentiment from tweet text and could serve as a foundation for automated sentiment monitoring. While there is room for improvement, the model successfully captures meaningful sentiment patterns and performs substantially better than the baseline approach.

8.2 Classification Report.

The classification report evaluates performance for each sentiment class individually. This helps identify which sentiment categories are classified accurately and which remain challenging for the model.

Classification Report

In [125…

```
# Generate classification report for the holdout test set
```

```
print(
    classification_report(y_test,y_pred )
)
```

```
              precision    recall  f1-score   support

    Negative       0.53      0.27      0.36       114
     Neutral       0.70      0.81      0.75      1075
    Positive       0.60      0.51      0.55       594
     Unclear       0.00      0.00      0.00        31

    accuracy                           0.67      1814
   macro avg       0.46      0.40      0.42      1814
weighted avg       0.65      0.67      0.65      1814
```

The model performed best on the Neutral sentiment class, achieving an F1-score of 0.75, while performance on the Positive class was moderate (F1-score = 0.55). The model struggled to identify Negative tweets (F1-score = 0.36) and failed to classify the Unclear class, likely due to the limited number of training examples for these minority classes.

The difference between the weighted F1-score (0.65) and macro F1-score (0.42) indicates that the model performs substantially better on the majority classes than on the minority classes. This pattern is consistent with the class imbalance observed during EDA.

8.3 Confusion Matrix

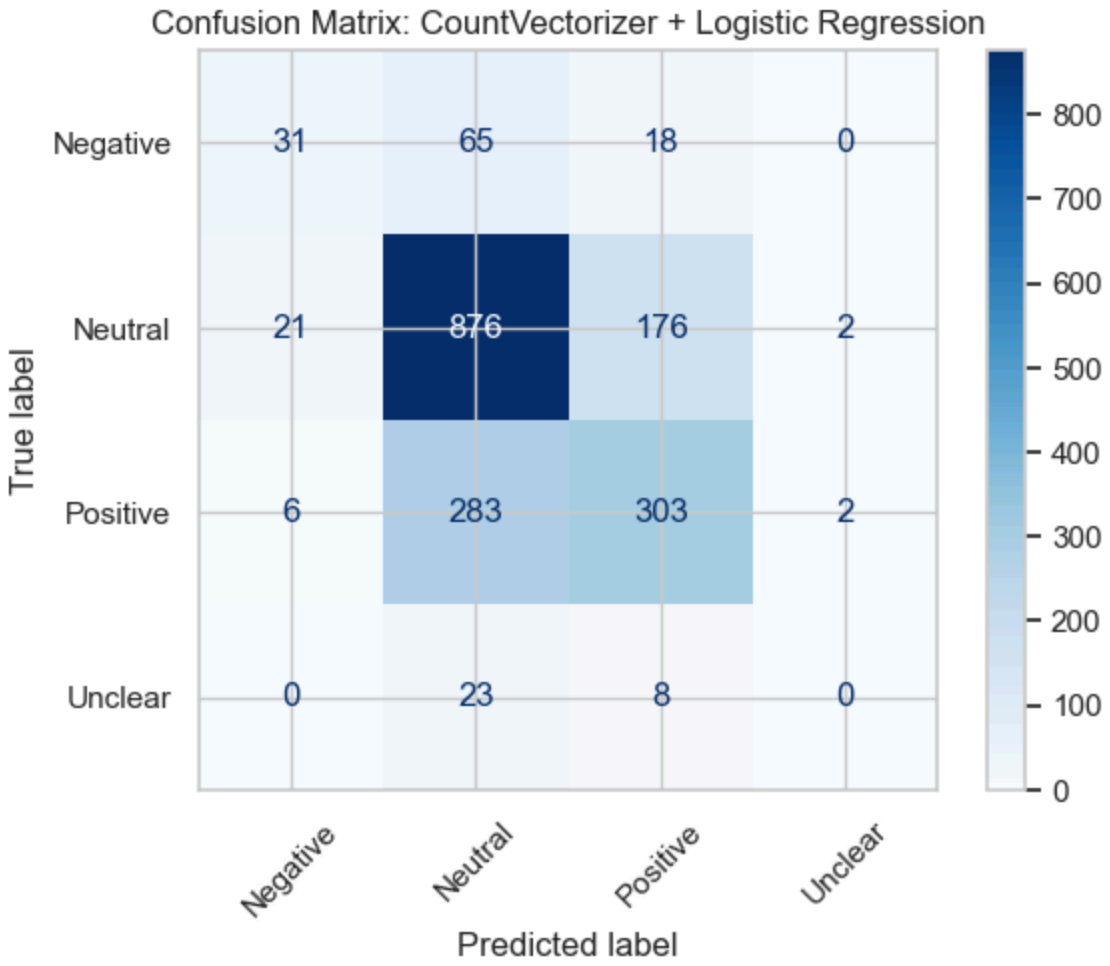
While the classification report summarizes model performance, the confusion matrix provides a detailed view of prediction outcomes. It shows how often each sentiment class was correctly classified and which classes were commonly confused with one another.

This helps identify specific areas where the model performs well and where misclassification occurs.

Confusion Matrix

In [126…

```
# Import visualization libraries
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

# Create confusion matrix
ConfusionMatrixDisplay.from_predictions(y_test,y_pred,cmap='Blues',xticks_rotation=45)

# Add title
plt.title('Confusion Matrix: CountVectorizer + Logistic Regression')

# Display plot
plt.show()
```



The confusion matrix shows that the model was most effective at identifying Neutral tweets, correctly classifying 875 out of 1,075 instances. This aligns with the high recall (0.81) observed in the classification report.

A large proportion of Positive tweets were correctly classified (303), although many were misclassified as Neutral (283), indicating overlap between these sentiment categories.

The model struggled with Negative tweets, correctly identifying only 31 out of 114 instances, with most being incorrectly classified as Neutral. Similarly, all Unclear tweets were misclassified as either Neutral or Positive, explaining the zero precision, recall, and F1-score for this class.

Overall, the model demonstrates a tendency to predict the dominant Neutral class, particularly when sentiment signals are weak or ambiguous.

9. Model Interpretability

To understand how the final model predicts sentiment, the most influential words associated with each sentiment class are examined. This helps identify the language patterns driving sentiment predictions and provides insight into the terms most strongly associated with customer opinions.

9.1 Most Influential Words

Identification of the words that contribute most strongly to the model's sentiment predictions. Understanding these influential terms improves model transparency and helps validate whether the model is learning meaningful sentiment patterns.

Extract Top Words for Each Sentiment Class

In [127…

```
# Get feature names from CountVectorizer
feature_names = count_vectorizer.get_feature_names_out()

# Create dataframe of coefficients
coef_df = pd.DataFrame(
    final_model.coef_.T,
    index=feature_names,
    columns=final_model.classes_
)

# Display top positive words for each sentiment class
for sentiment in coef_df.columns:

    print(f"\nTop 10 words for {sentiment}:")

    print(
        coef_df[sentiment]
        .sort_values(ascending=False)
        .head(10)
    )
```

```
Top 10 words for Negative:
headache    1.820494
fail        1.647107
hate        1.580338
suck        1.325044
button      1.318876
long        1.075647
enough      1.047631
mad         1.047405
launched    1.043680
lame        1.021939
Name: Negative, dtype: float64

Top 10 words for Neutral:
joke            0.977317
quotsxswquot    0.962794
torrent         0.930509
sxsws           0.927632
screw           0.921878
extend          0.908166
toy             0.896807
amp             0.895920
click           0.855428
traveling       0.840703
Name: Neutral, dtype: float64

Top 10 words for Positive:
smart       1.632218
cool        1.624684
loving      1.420317
hot         1.411049
fun         1.373371
great       1.269825
woot        1.217457
musthave    1.195142
```

```
excited     1.175193
genius      1.174636
Name: Positive, dtype: float64

Top 10 words for Unclear:
nut                       1.380386
home                      1.115512
en                        0.968291
fragmentationgtgoogle     0.891491
form                      0.891030
ave                       0.880792
erects                    0.874375
arrested                  0.867624
rww                       0.846597
staring                   0.812633
Name: Unclear, dtype: float64
```

The most influential words identified by the model align closely with their associated sentiment classes. Negative sentiment was strongly associated with words such as "headache", "fail", "hate", "suck", and "lame", which reflect frustration and dissatisfaction. Positive sentiment was driven by terms such as "smart", "cool", "loving", "great", and "excited" indicating favorable customer perceptions.

The Neutral and Unclear classes were associated with more general or context-dependent terms that do not convey strong emotional sentiment. The results suggest that the model learned meaningful language patterns that are consistent with expected sentiment expressions.

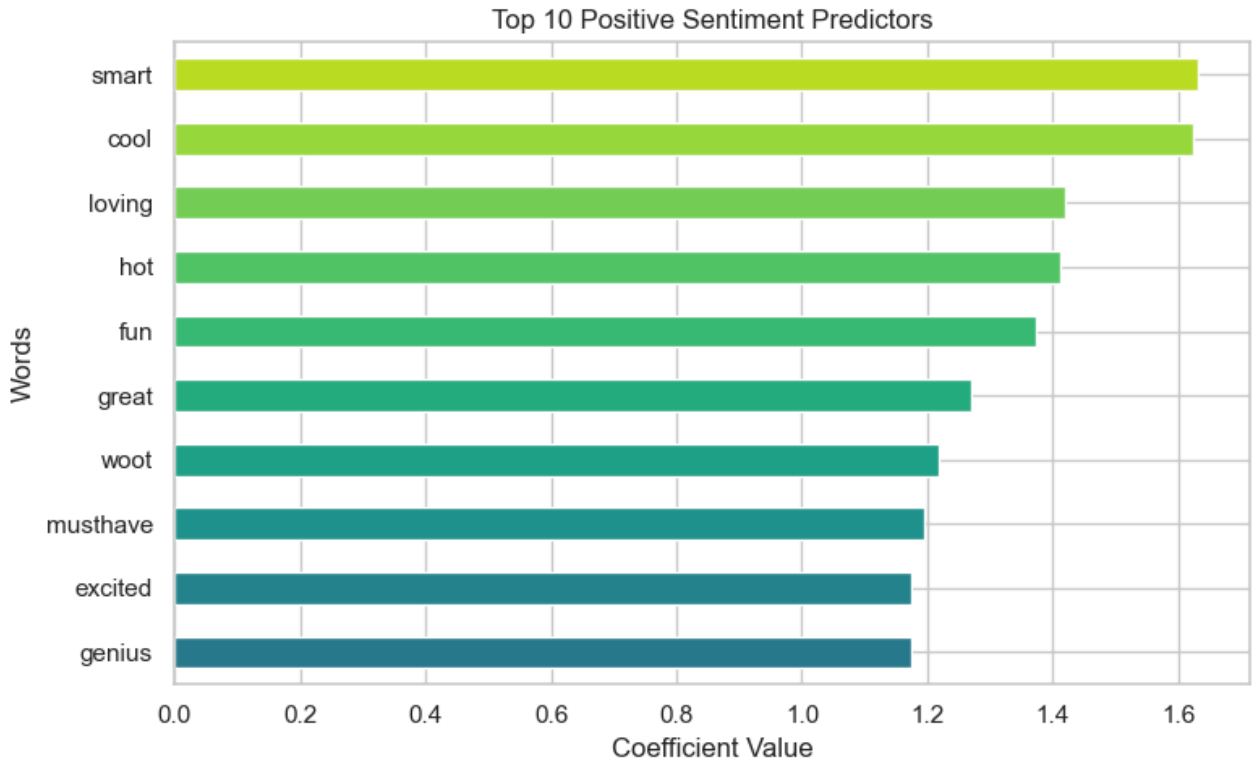
9.2 Top Positive Predictors

Visualization of words that most strongly contribute to positive sentiment predictions.

Visualize Top Positive Words

In [128…

```
# Extract top 10 positive predictors
top_positive = (
    coef_df['Positive']
    .sort_values(ascending=True)
    .tail(10)
)

# Create visualization
plt.figure(figsize=(8, 5))

top_positive.plot(
    kind='barh',
    color=plt.cm.viridis(np.linspace(0.4, 0.9, len(top_positive)))
)

plt.title('Top 10 Positive Sentiment Predictors')
plt.xlabel('Coefficient Value')
plt.ylabel('Words')

plt.tight_layout()
plt.show()
```



The visualization highlights the words most strongly associated with positive sentiment.

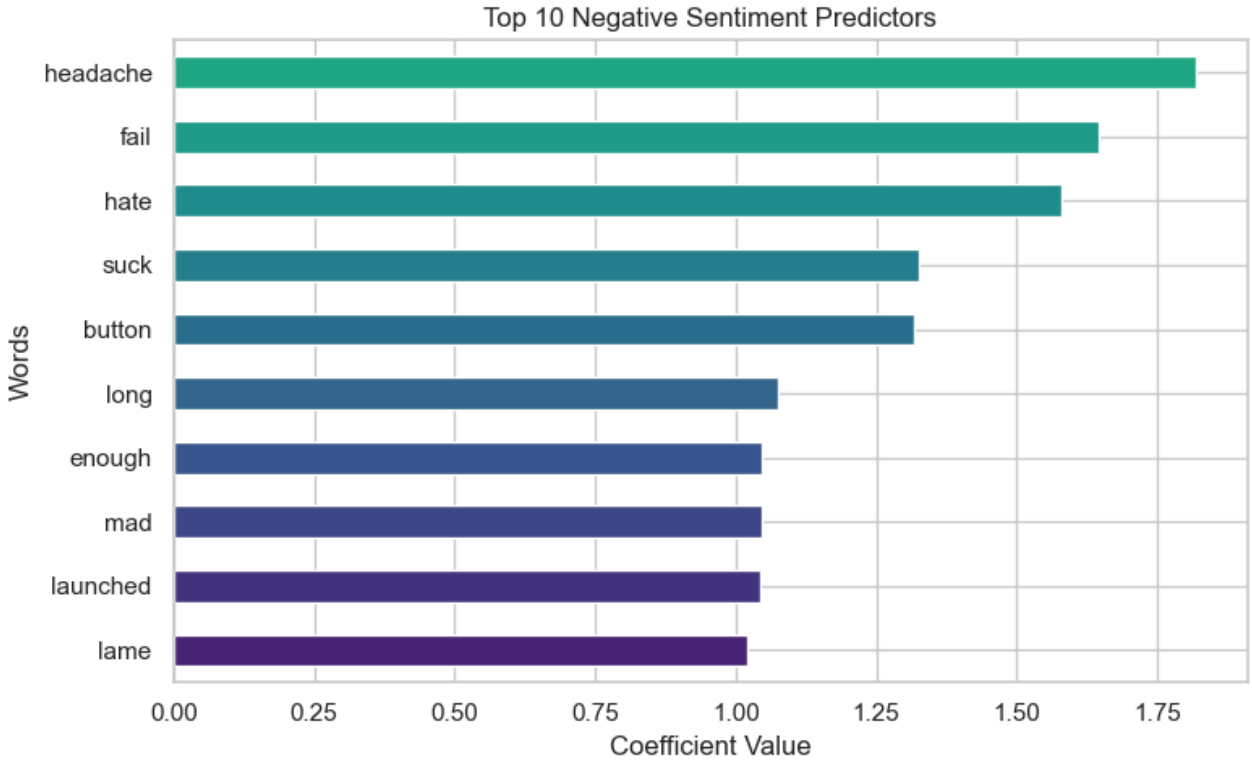
9.3 Top Negative Predictors

Visualization of words that most strongly contribute to negative sentiment predictions.

Visualize Top Negative Words

In [130…

```
# Extract top 10 negative predictors
top_negative = (
    coef_df['Negative']
    .sort_values(ascending=True)
    .tail(10)
)

# Create visualization
plt.figure(figsize=(8, 5))

top_negative.plot(
    kind='barh',
    color=plt.cm.viridis(np.linspace(0.1, 0.6, len(top_negative)))
)

plt.title('Top 10 Negative Sentiment Predictors')
plt.xlabel('Coefficient Value')
plt.ylabel('Words')

plt.tight_layout()
plt.show()
```



Based on the findings, organizations should continuously monitor customer sentiment across social media platforms to identify emerging issues and opportunities. Particular attention should be given to products associated with higher levels of negative sentiment, as recurring complaints may indicate areas requiring product or service improvements.

Organizations should also leverage positive feedback to identify features, campaigns, or experiences that resonate with customers and contribute to favorable brand perception.

10. Business Insights, Recommendations, Limitations, and Next Steps

10.1 Business Insights

Business Question 1: What is the overall sentiment of customer conversations?

The analysis revealed that Neutral sentiment dominated customer conversations, accounting for the majority of tweets within the dataset. This suggests that most users engaged in informational discussions, event participation, product mentions, and general commentary rather than expressing strong emotional opinions. Positive sentiment was the second most common category, indicating that many customers shared favorable experiences and enthusiasm toward products and services. In contrast, Negative and Unclear sentiments represented a relatively small proportion of the conversation, highlighting that dissatisfaction was present but not widespread.

From a business perspective, this suggests that customer conversations were generally balanced, with positive experiences outweighing negative ones. However, even a relatively small volume of negative sentiment can contain valuable insights into customer frustrations and opportunities for improvement.

Business Question 2: Which products are associated with the most positive and negative sentiment?

The sentiment distribution analysis revealed notable differences across product categories. While most products were associated with predominantly positive sentiment, iPhone-related tweets exhibited the highest proportion of negative sentiment among the major product groups.

This finding suggests that customers discussing the iPhone were more likely to express frustrations, complaints, or concerns compared to users discussing other products. Although the overall sentiment remained largely positive or neutral, the elevated concentration of negative sentiment around the iPhone indicates potential issues related to user experience, expectations, product performance, or product launches.

Organizations can use this type of analysis to identify products requiring additional attention, prioritize customer feedback investigations, and monitor sentiment trends following product updates or marketing campaigns.

Business Question 3: Can customer sentiment be predicted from tweet text?

The modeling results demonstrated that customer sentiment can be predicted from tweet text with reasonable accuracy. Among the evaluated models, CountVectorizer combined with Logistic Regression achieved the strongest performance, producing a weighted F1-score of 0.65 on unseen test data.

The relatively small difference between cross-validation performance and test-set performance suggests that the model generalizes well and is capable of identifying meaningful sentiment patterns beyond the training dataset.

Although performance varied across sentiment classes, the results demonstrate that machine learning can be used to automate sentiment classification and support large-scale analysis of customer conversations.

Business Question 4: What language drives customer sentiment?

The model interpretability analysis provided valuable insight into the language associated with positive and negative customer experiences.

Positive sentiment was strongly associated with terms such as *smart*, *cool*, *loving*, *great*, *excited*, and *genius*. These words reflect customer enthusiasm, satisfaction, and appreciation for products and experiences.

Negative sentiment was associated with words such as *headache*, *fail*, *hate*, *suck*, *mad*, and *lame*. These terms indicate frustration, dissatisfaction, usability concerns, and negative customer experiences.

The presence of these intuitive sentiment indicators increases confidence that the model learned meaningful language patterns rather than relying on arbitrary features. Furthermore, these insights provide organizations with a clearer understanding of how customers express both positive and negative perceptions online.

Business Question 5: What challenges were identified in sentiment classification?

While the model achieved strong overall performance, the evaluation results revealed challenges associated with minority sentiment classes. The model performed well on Neutral and Positive tweets but struggled to identify Negative and Unclear sentiments consistently.

This behavior was largely driven by class imbalance within the dataset, where Neutral tweets substantially outnumbered Negative and Unclear tweets. As a result, the model tended to classify ambiguous tweets as Neutral.

This finding highlights the importance of collecting balanced datasets and monitoring performance beyond overall accuracy when developing sentiment classification systems.

10.2 Recommendations

Based on the findings of this study, organizations should implement continuous sentiment monitoring to track customer perceptions across products and services. Automated sentiment classification can provide early warning signals for emerging customer concerns and help prioritize support and product improvement efforts.

Particular attention should be given to products associated with elevated levels of negative sentiment. Investigating recurring complaints and monitoring sentiment following product releases can help identify issues before they significantly impact customer satisfaction.

Organizations should also leverage positive sentiment insights to understand which product features, campaigns, and customer experiences generate enthusiasm and engagement. Identifying the drivers of positive sentiment can support future product development and marketing strategies.

Finally, sentiment analysis should be incorporated alongside traditional customer feedback channels to provide a more comprehensive understanding of customer needs and perceptions.

10.3 Limitations

Several limitations should be considered when interpreting the results of this analysis.

First, the dataset exhibited substantial class imbalance, with Neutral sentiment significantly outnumbering Negative and Unclear sentiments. This imbalance influenced model performance and contributed to weaker classification results for minority classes.

Second, some non-informative tokens remained within the dataset after preprocessing and appeared among influential predictors for certain sentiment categories. Additional text cleaning and normalization may further improve model quality.

Third, the analysis was conducted using historical tweets collected during a specific period. Customer perceptions, language patterns, and product discussions may evolve over time, limiting the generalizability of the findings to future contexts.

Finally, sentiment classification models are limited in their ability to fully capture sarcasm, irony, humor, and contextual nuances that are often present in social media conversations.

10.4 Next Steps

Future work should focus on improving the representation of minority sentiment classes through additional data collection and class balancing techniques. Increasing the number of Negative and Unclear examples would likely improve model performance and enhance the reliability of sentiment predictions across all classes.

Although an advanced ensemble model (Random Forest) was evaluated, future research could explore more sophisticated approaches, such as gradient boosting techniques and transformer-based language models, to determine whether contextual language understanding improves sentiment classification performance.

Finally, the developed model could be integrated into a real-time sentiment monitoring dashboard, enabling organizations to continuously track customer sentiment, identify emerging trends, and support data-driven decision-making.

11. References

CrowdFlower. (2013). *Twitter sentiment dataset for Apple and Google products*. Retrieved from https://data.world/crowdflower/brands-and-product-emotions

McKinney, W. (2023). *pandas documentation*. Retrieved from https://pandas.pydata.org/

NumPy Developers. (2024). *NumPy documentation*. Retrieved from https://numpy.org/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. Retrieved from https://scikit-learn.org/

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. Retrieved from https://www.nltk.org/

Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90–95. Retrieved from https://matplotlib.org/

Waskom, M. L. (2021). *Seaborn: Statistical data visualization*. Journal of Open Source Software, 6(60), 3021. Retrieved from https://seaborn.pydata.org/

Mueller, A. (2024). *WordCloud for Python documentation*. Retrieved from https://amueller.github.io/word_cloud/